

AI-powered Auto-Completion Tool for Solidity Code

Research Project

AI-powered Auto-Completion Tool for Solidity Code

Fabian Hensel

March 31, 2025

submitted to the Institute for Data Engineering
at the Hamburg University of Technology.

Supervisors:

Avik Banerjee
Institute for Data Engineering
TU Hamburg

Prof. Dr.-Ing. Stefan Schulte
Institute for Data Engineering
TU Hamburg

Hamburg University of Technology

Institute for Data Engineering

Am Schwarzenberg-Campus 1

21073 Hamburg

Germany

Fabian Hensel

546702

AI-powered Auto-Completion Tool for Solidity Code

March 31, 2025

to the reader

Contents

Abstract	ix
1 Introduction	1
2 Background	3
2.1 The Ethereum Blockchain Platform	3
2.2 Transformers	5
2.3 Large Language Models	7
2.4 AI-Powered Auto-Completion	8
3 Related Work	11
4 Fine-tuning an LLM	13
5 Implementation	17
5.1 Data Gathering	17
5.2 Data Preprocessing	18
5.3 Model Fine-tuning	30
6 Evaluation	37
7 Conclusion	43

Abstract

With the growing adoption of blockchain technology, Solidity has become the dominant language for smart contract development. However, writing secure, efficient, and syntactically correct Solidity code presents challenges due to the language's unique constraints and susceptibility to vulnerabilities. This research investigates the fine-tuning of a Large Language Model (LLM) for automatic Solidity code completion, aiming to improve developer productivity and code reliability.

A pretrained code LLM is fine-tuned using a curated dataset of high-quality Solidity code, allowing it to learn domain-specific syntax, common coding patterns, best practices, and security considerations. Each Solidity file in the dataset undergoes a comprehensive preprocessing pipeline. The fine-tuning process involves transfer learning techniques to adapt the model's understanding to Solidity's distinct structure and semantics. The effectiveness of the fine-tuned model is assessed by comparing it against the base model using perplexity, the Bilingual Evaluation Understudy (BLEU) score, and the Metric for Evaluation of Translation with Explicit ORdering (METEOR) score, evaluating improvements in code fluency, syntactic accuracy, and contextual relevance.

Experimental results reveal that fine-tuning significantly enhances the model's ability to generate syntactically correct and contextually appropriate Solidity code completions. It also shows that the model is to some extent capable of generating secure code or annotations on vulnerable lines of code. These findings underscore the potential of specialized LLMs in aiding smart contract development, reducing manual coding effort and minimizing syntax errors and security issues. This study contributes to the growing field of AI-assisted programming, offering insights into the application of fine-tuned language models for domain-specific tasks in blockchain development.

1

Introduction

Blockchain is a comparatively young technology that was first introduced in 2008 with the white paper on the cryptocurrency Bitcoin¹ [Nak08]. Further developments of this technology led to second-generation blockchains, which can also execute code. There are many different blockchain protocols. The first and most famous one is Ethereum², which was introduced in 2014 in [But14]. Ethereum can be understood as a decentralized “world computer” where everyone connected to it can write code. This code is provided in the form of smart contracts. These are small computer programs that are executed by the so-called Ethereum Virtual Machine (EVM), which runs on all nodes of the Ethereum network. The execution of this code is triggered when a transaction is sent to a smart contract. Special programming languages are required to write these smart contracts. Different languages exist for this purpose, e.g. Solidity, Vyper, Yul, Fe, etc. By far the best known is Solidity. It is a high-level language most comparable to JavaScript. Both share many syntactic and conceptual similarities, as [Edi24] shows.

Anyone who has already programmed with a high-level programming language knows how complex this is. The same applies to Solidity, which also offers many different and complex functionalities. This is a double-edged sword, as more functionalities naturally lead to more and better ways of achieving a desired goal, but also to more comprehension problems and mistakes in general [McC04]. This can limit the productivity of the programmer and the efficiency of the code. Efficiency plays a very important role when programming smart contracts on the EVM, as higher execution times or code sizes lead to higher gas consumption. The EVM introduces the concept of gas as remuneration for validators who mine blocks and validate blockchain transactions. There is also an upper limit on the gas consumption of a smart

¹<https://bitcoin.org/en/>

²<https://ethereum.org/de/>

contract to avoid endless loops, which would otherwise render Ethereum unusable. To deploy a contract one needs to pay a gas fee with Ethereum's cryptocurrency Ether [AW19, p. 312 et seq.]. In other words, the better the efficiency, the less one has to pay.

Another consequence is security gaps in smart contracts due to incorrect programming. Since smart contracts sometimes manage and control large amounts of money, this is a very serious problem. There are many examples of contracts that have been hacked due to mistakes by the programmer. A famous example is the decentralized autonomous organization (DAO) attack [Sie16], in which attackers managed to steal (at that time) 150 million dollars from a contract. Of course, this is a worst-case scenario, but it shows the utmost importance of using secure code in smart contracts. Many of Solidity's security vulnerabilities have already been discovered and can be avoided by using best practices and appropriate libraries (e.g., the SafeMath³ library to avoid arithmetical overflows or underflows). [Edi23] shows some common Solidity vulnerabilities and corresponding prevention mechanisms.

For general-purpose programming languages, there are code assistance tools such as GitHub Copilot⁴ that help programmers increase their productivity and write efficient code. For example, according to [Kal22], software developers who use Copilot are 55% faster when writing code than developers who do not use it. They also have a 8% higher rate of completing a task. This is done using techniques such as automatic code completion and context-aware code suggestions. Copilot also helps to improve the security of the code by using secure coding patterns and best practices adapted to the respective programming language. Furthermore, it can help to find common vulnerabilities in the code and suggest appropriate fixes. However, it should not be used for a comprehensive security analysis [Doc25].

Similar to Copilot, an Artificial Intelligence (AI)-powered code assistant specifically tailored for Solidity programming can significantly enhance the development of smart contracts. This work focuses on fine-tuning a state-of-the-art transformer-based LLM to enable automatic code completion for Solidity. The process involves adapting an existing code LLM to understand the unique syntax, structures, coding best practices, and security considerations of Solidity. This requires the creation of a specialized dataset with a comprehensive preprocessing pipeline. By fine-tuning the model on this dataset, the goal is to improve the accuracy, relevance, and security of code completions, ultimately enhancing the developer experience and contributing to the development of more secure and efficient smart contracts.

³<https://docs.openzeppelin.com/contracts/2.x/api/math>

⁴<https://github.com/features/copilot>

2

Background

This chapter sets out the background for further understanding the implementation phase. Section 2.1 provides a short introduction to the blockchain technology, in particular to Ethereum. It also explains smart contracts and the Solidity programming language. Section 2.3 explains what an LLM is. Last but not least section 2.4 explains what are code assistance tools, in particular automatic code completion.

2.1 The Ethereum Blockchain Platform

Ethereum is based on blockchain technology, which, according to [SAP24], is designed to enable a secure, transparent, and decentralized exchange of information within a network. By eliminating the need for central authorities, it fosters trust among participants. All transactions are visible and traceable, ensuring transparency, while cryptographic methods protect data from manipulation. The immutability of blockchain guarantees that data cannot be altered without consensus, ensuring consistency. Additionally, automated processes reduce the reliance on intermediaries, making operations more efficient and cost-effective. Blockchain is composed of a data structure and a system for managing this data structure [FS19, p. 80 et seq.]. The data structure consists of chronologically linked blocks, which itself represent structured datasets. These blocks store transactions created and signed (with asymmetric cryptographic systems) by participants and are chained together with cryptographic hash functions. The blockchain system is a network of many distributed nodes, where each node contains a copy of the entire blockchain. All nodes have equal rights and can communicate directly with each other, i.e. the network functions as a peer-to-peer network.

The development phase of blockchains can be divided into first generation

blockchains and second generation blockchains. The former is used for the decentralized recording of transactions, especially for cryptocurrencies. So the main goal of such blockchains is to establish a decentralized digital monetary system. Prominent examples are Bitcoin and Litecoin¹. The latter extends original blockchain technology by providing a platform for programmable decentralized application (DApp) and smart contracts. These platforms therefore support applications that go far beyond cryptocurrencies. Prominent examples are Ethereum, Solana², Cardano³, and Sui⁴. They all have certain characteristics and use different programming languages to create smart contracts. Ethereum's main programming language is Solidity, which was developed solely for programming smart contracts. Rust is mainly used for Solana, Plutus for Cardano, and Move for Sui.

Ethereum can be described as a transaction-based state machine. This means that Ethereum exists in a single global state, which transitions to a new, updated state through transactions. Viewed this way, Ethereum can be considered a world computer, as even the smallest action by an external actor causes the entire Ethereum state to update for every participant in the network [FS19, p. 113]. The heart of Ethereum is the EVM (and the reason for calling Ethereum a world computer). The role of the EVM is the deployment and execution of smart contracts. A local instance of the EVM exists on every node in the Ethereum network. Each of these instances operates with the same initial state and produces the same final state. It computes the update to the state resulting from the execution of a smart contract [AW19, p. 118]. The EVM can also be compared to the Java Virtual Machine (JVM). The JVM abstracts the underlying hardware of a system, providing compatibility with many different systems. Code written in programming languages like Java is compiled into JVM bytecode, which is then executed by the JVM. Similarly, the EVM executes bytecode compiled specifically for it. This bytecode is generated during the compilation of source code written in a smart contract programming language such as Solidity, using its respective compiler [AW19, p. 299].

The cryptocurrency used within the Ethereum network is called Ether (abbreviated as ETH). Ether serves as a means of payment for executing smart contracts and DApps within the network [AW19, p. 15]. It is therefore fundamentally different from Bitcoin, whose sole purpose is to be a currency. Another important component used for Ethereum is the so-called gas. As the EVM is Turing-complete, it is subject to the halting problem. This means that it is not possible to decide whether an algorithm (smart contract) will terminate. To avoid this, one purpose of gas is to specify the maximum amount of resources to be used by the EVM for a program before execution is aborted [AW19, p. 312 et seq.].

¹<https://litecoin.org/>

²<https://solana.com/de>

³<https://cardano.org/>

⁴<https://sui.io/>

Smart Contracts

Smart contracts are self-executing programs between multiple parties that can be created using various programming languages [Ava25]. The choice of language depends on the underlying blockchain platform and the programmer's preferences. They contain predefined conditions and actions that are executed automatically as soon as the corresponding conditions are met. In the context of previous section, these are small computer programs that run on a decentralized world computer (the EVM). As they are based on the blockchain technology, they also use its advantages, such as transparency and counterfeit protection [AW19, p. 127 et seq.]. There are a plenty of use cases for it such as decentralized finance (DeFi) contracts, fungible token contracts, non-fungible token (NFT) contracts, gaming contracts, insurance contracts, and so on [Dil24].

Solidity

Solidity is the most commonly used programming language for creating smart contracts on the Ethereum network [Cha24]. It is an imperative, curly-bracket, statically typed language that supports familiar object-oriented mechanisms, such as inheritance, libraries, and user-defined types. Solidity was influenced by several programming languages, with the most significant influences coming from JavaScript, C++, and Python. Its syntax and features are designed to simplify the implementation of smart contracts, allowing developers to define and automate interactions on the blockchain securely. Additionally, Solidity provides built-in support for handling blockchain-specific features such as addresses, events, and Ethereum's gas model, making it highly suited for decentralized applications [Eth24].

2.2 Transformers

Today, LLMs are mostly based on the transformer technology, which was introduced by [VSP+17] in 2017. Transformers are a specific kind of neural network and were a milestone for LLMs. Unlike the previously used technique, recurrent neural networks, which work purely sequentially, they enable parallel processing of entire sections of text thanks to their self-attention mechanisms. This mechanism assigns importance to each token in a sequence, allowing every token to interact with any other token. This interaction enables the model to capture long-range dependencies and understand the overall context more deeply.

Equation 2.1 shows the attention mechanism. Each token is represented by three vectors, namely the query (Q), key (K), and value (V) vectors. To obtain the attention scores, i.e. how much importance one token should give to another token, the dot product of the query vector of one token and the key

vector of all tokens is computed. This attention score is then divided by the square root of the dimension of the key vectors, which is required for stable training. The softmax function returns probabilities. These probabilities assign weights to each token's value vector, ultimately generating a representation that incorporates contextual information for every token [ZLL+21].

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.1)$$

Transformers consist primarily of two components, namely the encoder and the decoder. The encoder processes input sequences into bidirectional contextualized representations, while the decoder generates output sequences autoregressively using the encoder outputs. The intended purpose of the transformer technology was language translation, which is a type of Sequence to Sequence (S2S) modeling and therefore uses both the encoder and the decoder. Hence, S2S models are also called encoder-decoder models. On the other hand, only the encoder is used for Masked Language Model (MLM)s and only the decoder for Causal Language Model (CLM)s, which is why they are also referred to as encoder-only models and decoder-only models [Fac22].

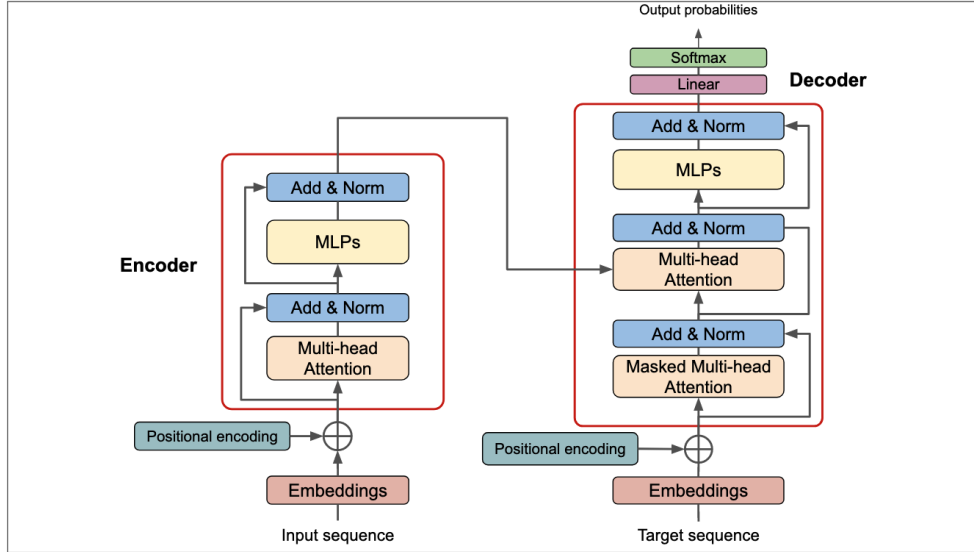


Figure 2.1: The transformer architecture. Source: www.luxiangdong.com

The encoder and decoder themselves also consist of several subcomponents as shown in figure 2.1, namely input/output embedding, positional encoding, (masked) multi-head attention, layer normalization, residual connections and feedforward network layers. The embedding converts each token into a high-dimensional numerical vector representation, which captures semantic information such as the relation of two tokens. The so-called embedding matrix has an entry for each token of the vocabulary of the tokenizer with which the dataset was tokenized for training. For example, the tokenizer

used for Generative Pretrained Transformer (GPT)-3 has a vocabulary size of 50,257 tokens, i.e. the matrix can be seen as a list of 50,257 high-dimensional vectors. Positional encodings are required to encode the position of each token within the sequence. Multi-head attention simply performs multiple attention “heads” instead of a single attention operation. Each head applies a unique set of linear projections to queries, keys, and values, enabling the model to capture different aspects of token relationships. Its primary purpose is to stabilize the training process and enhance convergence by normalizing the activations across the features of each token [Fer24].

2.3 Large Language Models

LLMs [NKQ+24] are a machine learning (ML) technique used in the context of natural language processing. They use statistical methods and deep neural networks that are trained in a self-supervised manner (i.e. the data are not labeled by humans, as the objective is automatically computed from the model’s inputs) on a huge corpus of text data to develop a statistical understanding of it. This is the first step in an LLM training process, called pretraining, which is carried out using the MLM paradigm (auto-encoding models), CLM paradigm (autoregressive models) or S2S model paradigm.

MLMs predict a masked word in a sentence using bidirectional context. For example, for the input »*The [mask] is shining, what a beautiful day*«, the model should predict *sun*. The model predicts the missing word with the left-hand and right-hand context of the sentence. MLMs are well suited for tasks such as named entity recognition, where the goal is to identify entities, e.g. people or locations, in text (i.e. for tasks that require an understanding of the full sentence). Google’s Bidirectional Encoder Representations from Transformers (BERT) model [DCLT19] is an example of an MLM.

CLMs predict the next word in a sentence, given only the preceding words. For example, for the input »*The sun is shining, what a beautiful . . .*«, the model should predict *day*. The model therefore predicts the word with the left-hand context of the sentence. Everything on the right-hand side of the sentence has no influence on the prediction. CLMs are ideal for tasks such as text generation (e.g. chatbots). OpenAI’s⁵ GPT [BMR+20] models are CLMs. For example, GPT-3 was pretrained with 570 GB of data, mainly obtained with web crawling and from books, web texts and Wikipedia [Iye22]. The data are filtered, i.e. low-quality content or duplicates are removed and formatting adjustments are made. The raw dataset is much larger at around 45 TB. A frequently used metric to indicate the size of the dataset is the number of tokens, which were about 410 billion for GPT-3 [Res25]. Tokens are words or sub-words of a text that were obtained after tokenization of the dataset.

S2S models transform an input sequence into a corresponding output se-

⁵<https://openai.com/>

quence. For example, for the translation of the input »*The sun is shining, what a beautiful day*« to German, the model should predict *Die Sonne scheint, was für ein schöner Tag*. The output sequence is reconstructed in an autoregressive manner. The objective for the pretraining of such models can be the same as for MLMs or CLMs, but is usually more complex. S2S models are not only well suited for tasks such as language translation, but also for text simplification or summarization. Google’s T5 model [RSR+20] is an example of an S2S model.

To achieve better quality in specific language tasks such as language translation or text processing (e.g. summarize articles, analyze books or extract information from an article), in a second step the model needs to be fine-tuned with a specific dataset for these tasks. Now in a supervised manner, i.e. the data is labeled by humans [Fac22]. The process of pretraining a model in a first step with unlabeled data and fine-tuning it in a second step with labeled data is not to be confused with semi-supervised learning. The model is trained here in a single training phase with both labeled and unlabeled data [PK08].

The term “large” refers to the fact that these models often include billions of parameters, which are also called weights and learned/updated with a process known as backpropagation. With backpropagation the new weights of the network are computed after a feedforward step with the gradients of the loss function with respect to the weights. The loss function defines how close the models predictions are to the real labels. To find a local minimum of the loss function, an optimization algorithm called gradient descent is used. Due to the large number of weights, the search space is very large, making it very difficult to find the global minimum within it. Gradient descent adjusts the weights by moving them with a step size defined before the start of training, the so-called learning rate, in the direction of the steepest decrease (where the computed gradients point to) of the loss function to find its minimum ([Nie19], [ZLL+21]).

GPT-3, for example, has 175 billion parameters. For the pretraining, a supercomputer with more than 285,000 CPU cores and 10,000 Nvidia V100⁶ GPUs was developed [Ala20]. It’s easy to imagine how much this costs and how much energy is required to run it. The carbon footprint is therefore not negligible either, and newer models, such as GTP-4 [OAA+24], are becoming even larger. As the models become ever larger and more resource-intensive, the carbon footprint also increases considerably [SGM19].

2.4 AI-Powered Auto-Completion

Typical examples for code assistance tools include automatic code completion, syntax highlighting, syntax checking, linting, code folding, code formatting, debugging tools, static code analysis, and version control tools (e.g. Git). Syn-

⁶<https://www.nvidia.com/de-de/data-center/tesla-v100/>

tax highlighting, as the name suggests, is used to highlight language specific keywords (datatypes, control flow keywords, object-oriented programming keywords, etc.), identifiers, comments, literals, and syntax elements in different colors to make the code more readable. To detect these language elements in the code, a lexical analysis is performed (the first stage of a compiler, known as lexer) [Mey21, p. 79 et seq.]. Syntax checking checks for syntactically correct code in real time, e.g. it detects missing semicolons, wrong keywords, or unclosed brackets. It is performed with the second stage of a compiler, namely the syntactical analysis (known as parser) [Mey21, p. 133 et seq.]. The linter analyzes the code for style guide and best practice issues. It is also able to detect potential bugs. Static code analysis tools are used to detect security vulnerabilities or errors without executing the code. To do this, they use the so-called abstract syntax tree of the code [FGG19].

Automatic code completion suggests or automatically completes/fills in code given a partial input from the developer. There are two common ways of implementing it, namely rule-based or ML-based [Gru21]. The latter was in earlier approaches implemented with ML methods such as decision trees or support vector machines. Today, it is mostly implemented with transformer-based LLMs. In this case, however, they are pretrained on a huge corpus of code, usually from many different programming languages, and are fine-tuned with suitable datasets for code-specific tasks such as code generation with text prompts or automatic code completion. Examples for code LLMs are Code Large Language Model Meta AI (LLaMA) [RGG+24], CodeT5+ [WLG+23], StarCoder [LAZ+23], or DeepSeek-Coder [GZY+24]. LLMs for automatic code completion are usually trained autoregressively. This makes sense as they should be able to generate a new token from the previous tokens, i.e. complete the input.

Normally, developers do not only program in a strict word-for-word order. Often they write code and fill in a missing part in that code block. Therefore, it is also necessary to make the model learn to generate code given the context before and after a missing part. In order to achieve this and still use an CLM paradigm, a so-called fill-in-the-middle (FIM) [BJT+22] approach is applied. For FIM the code structure (e.g. a function) has to be transformed into a prefix, middle, and suffix part. Then the middle part is moved to the end, which means that the structure is now prefix + suffix + middle (a so-called PSM transformation). Another common structure for FIM is suffix + prefix + middle (a so-called SPM transformation). How many lines are within the middle part depends on the FIM ratio, which gives a percentage of all lines of the code block (e.g. of a functions body). According to [BJT+22], a common starting point for the FIM ratio is 0.5. Exactly which lines are moved is selected at random. It is worth mentioning that the selection of the middle part is not limited to lines. The split can also be made at token or character level. Which approach to choose depends on the task at hand. Finally, special FIM tokens must be inserted to show the model where the prefix starts, the suffix ends, and the missing part has to be inserted.

3

Related Work

This section describes related work that provides context and background for the current project. There already exist AI-assistance tools for Solidity. Ethereum's IDE Remix¹ has integrated an AI tool called RemixAI², which is based on their own LLM called Solcoder. According to the Remix AI tools documentation, with RemixAI it is possible to get an explanation of a contract, to generate a documentation corresponding to this contract, and to get an explanation of compile errors. There is also the opportunity to ask it to write a certain function using a prompt. Furthermore, they have a sub-project called RemixAI Copilot, which is used for automatic code completion and context-aware code suggestions. The RemixAI Copilot is at the time of writing (March 31, 2025) in the beta phase. GitHub's Copilot also supports Solidity.

There is a variety of LLMs for code. Some of these models have already been fine-tuned with a Solidity smart contract dataset. For example, users³ on Hugging Face have fine-tuned a Code LLaMA model with 7 billion parameters to generate Solidity smart contract code. Fine-tuning was performed using 4-bit quantized low rank adaption (QLoRA) and with a dataset of 6,003 GPT-generated human instruction and Solidity source code data pairs. The focus of this model is on the generation of code with text prompts. Another Hugging Face user⁴ example is a CodeT5+ LLM, which was fine-tuned for Solidity code generation with comments. In the author's opinion, however, the model is clearly under-trained due to a lack of GPUs for the fine-tuning.

Suitable datasets play a crucial in fine-tuning an LLM. There already exist Solidity smart contract datasets such as the dataset from [Ros22]. The purpose of this dataset is to provide a freely available, large-scale dataset for the de-

¹<https://remix.ethereum.org/>

²<https://remix-ide.readthedocs.io/en/latest/ai.html>

³<https://huggingface.co/AlfredPros/CodeLlama-7b-Instruct-Solidity>

⁴<https://huggingface.co/hululuzhu/solidity-t5>

tection and classification of vulnerabilities in verified Solidity smart contracts. According to the creator of the dataset, any smart contract in it was tested with the static analysis framework Slither⁵ [FGG19]. However, this dataset appears to be out of date, as Slither now detects at least one security vulnerability for almost all contracts labeled as secure. Other examples are the DISL dataset [MEBM24], which contains 3,298,271 Solidity smart contracts or the SmartBugs⁶ wild dataset [DFAC20], which contains 47,398 SmartBugs analyzed smart contracts. None of these datasets were preprocessed for fine-tuning (except for the mentioned security analysis).

All in all, recent work in AI-assisted Solidity development has made significant strides, particularly with the use of LLMs for code completion. However, these approaches typically rely on datasets that are not extensively preprocessed. In contrast, this work introduces a more sophisticated approach by fine-tuning an LLM specifically for Solidity, using a highly specialized and rigorous preprocessing pipeline. This pipeline includes not only Slither analysis for security vulnerability detection, but also incorporates Solhint⁷ for style and linting checks, Prettier⁸ for formatting, and a unique FIM transformation. Furthermore, special security tokens and annotations in form of comments are inserted into the dataset to enhance the model’s ability to generate secure code. This comprehensive approach improves the model’s performance in generating code that meets both structural and security requirements, setting it apart from recent efforts in the field.

⁵<https://github.com/crytic/slither>

⁶<https://github.com/smartbugs/smartbugs>

⁷<https://github.com/protofire/solhint>

⁸<https://prettier.io/>

4

Fine-tuning an LLM

This chapter presents the approach of implementing the fine-tuning of an LLM for the automatic code completion of Solidity code. Common methods and libraries used in this context are presented. Fine-tuning an LLM is a broad field, there are many methods to do this. Which one should be used depends on the specific task (such as summarization, translation, classification of text or code/text generation) and the available resources (computing power and storage).

Approach

As is usual with ML methods, the first step in creating a good model is to gather enough data for the specific task. This data can be primarily obtained from sources like GitHub¹ and Etherscan² with the corresponding APIs. Both contain thousands of different Solidity smart contract source codes. A high diversity of different contract types must be collected to obtain a rich dataset. The “raw” data must then be preprocessed in order to obtain a high-quality dataset and tokenized with a suitable tokenizer, such as Byte-Pair Encoding (BPE) [SHB16], WordPiece [SN12], or SentencePiece [KR18], for subsequent model fine-tuning. The choice of tokenizer depends on the one used for pretraining the chosen model.

For deep learning models, there is a trade-off between model size in terms of number of parameters and accuracy. A smaller model size generally means a lower accuracy, while a higher accuracy means a larger model size. It should be noted that this is an oversimplification, as the relationship between model size and accuracy is not strictly linear [Ski25]. Since the aim of the work is to create a model which is, among others, able to generate secure code, the

¹<https://github.com/>

²<https://etherscan.io/>

focus lies on the highest possible accuracy. As explained in chapter 1, incorrect predictions and the resulting potentially insecure code could have catastrophic consequences and should therefore be avoided at all costs. LLMs require a lot of training data to achieve high prediction accuracy [BMR+20], and the model size is usually, as the name suggests, very large. For example, even the “small” LLM Phi-2 [Jav18] by Microsoft has 2.7 billion parameters. Due to its comparatively small size, Phi-2 is also considered as a Small Language Model (SLM) [Mas23]. SLMs are generally a good alternative to LLMs if fewer computational/memory resources are available.

Training LLMs (and often even SLMs) from scratch is highly resource intensive. Therefore, in this project an approach called transfer learning [ZQD+21] is used. In other words, a foundation model, which is already pretrained on a diverse range of data, is fine-tuned with the collected dataset for the purpose of this work. So the second step of the LLM training process explained in section 2.3 will be done. Such a foundation model will be obtained from the machine learning platform Hugging Face³. There are many open-source foundation models that could be used (e.g. BERT, LLaMA [TLI+23] or Phi), but not all are equally suitable for fine-tuning to a specific task. When selecting the foundation model, the model’s capabilities (how well they fit the task of this work), the pretraining data and general limitations/biases present in the model must be considered [BHA+22]. Of great interest for this work are open-source foundation models, which are already code-specialized. For example, the already mentioned LLMs Code LLaMA (code-specialized version of LLaMA 2), CodeT5+, StarCoder, or DeepSeek-Coder.

The full fine-tuning of such foundation models still requires a lot of computational resources, as parameter updates lead to an even higher memory consumption. For example, to update the parameters of the LLM Phi-2 with stochastic gradient descent, the derivative of each parameter with respect to the cost function needs to be computed and stored. So, in other words, this corresponds to a doubling of memory consumption during fine-tuning. With Momentum, an improved optimizer compared to gradient descent, the memory consumption would be three times as high and with Adam [KB15], an even better optimizer, four times as high [Rud17]. Around 22 GB is required to store 5.4 billion float32 parameters. Therefore, parameter efficient fine-tuning (PEFT) [XXQ+23] methods such as low rank adaptation (LoRA) [HSW+22], vector-based random matrix adaption (VeRA) [KBA24], or QLoRA [DPHZ23] will be used. Google Colab⁴ provides a platform to carry this out. Colab makes it possible to use high-end graphic cards for a much faster fine-tuning procedure. There are also numerous alternatives such as Amazon’s AWS⁵ or Microsoft’s Azure⁶.

³<https://huggingface.co/>

⁴<https://colab.research.google.com/>

⁵<https://aws.amazon.com/de/machine-learning/>

⁶<https://azure.microsoft.com/en-us/products/machine-learning/>

Finding the best possible hyperparameters for fine-tuning a model is a very important step. Hyperparameters are everything that is set before the fine-tuning starts, such as the learning rate, batch size, or the LoRA settings. They have a high impact on the result (in terms of any evaluation metric) of the final model. Hence, a hyperparameter optimization approach is used, which includes the definition of a search space, search algorithm, and scheduler. The search space defines the hyperparameters to be optimized and their possible values from which the search algorithm should choose. The search algorithm searches/suggest hyperparameter configurations. Examples are grid search, random search, genetic algorithm (see [LL19] for an explanation and comparison of these three) and bayesian optimization [Fra18]. The scheduler does early stopping for non-promising trials. Examples are asynchronous successive halving (ASHA) [LJR+20], median stopping rule [GSM+24] and population based training [JDO+17]. For the implementation, libraries such as Ray Tune⁷, Optuna⁸, or Hyperopt⁹ could be used [Boh23].

To evaluate the performance of the fine-tuned model, benchmarks are used. These are also very useful in comparing the model with other models (or with its base model). As described in [CS24] LLM benchmarks usually consist of sample data, a set of tasks to test the model on specific skills, evaluation metrics and a scoring mechanism. For this project, the tasks would consist of automatically completing code sequences or filling gaps in the code and the specific skills to be tested could be structural correctness as well as secure and best practice code. There are various evaluation metrics for LLMs, which can be divided into intrinsic and extrinsic ones. Intrinsic evaluation metrics assess the model's performance directly by analyzing its predictions, typically by comparing a generated output with a reference output. Common intrinsic metrics for LLMs are perplexity, BLEU [PRWZ02] resp. CodeBLEU [RGL+20], METEOR [BL05], and Recall-Oriented Understudy for Gisting Evaluation (ROUGE) [Lin04]. The intended purpose of most of these metrics is not to evaluate generated code, but they can be used for that as well. Extrinsic metrics are used to evaluate the model's performance on real-world applications [Ver23]. Unlike intrinsic metrics, which focus mainly on lexical and syntactical correctness, with some limited assessment of semantics (the focus depends on the metric), extrinsic metrics usually also focus on the functional correctness of generated code. This is done, for example, by applying additional unit tests. A common extrinsic metric used in the context of code generation evaluation is Pass@k [CTJ+21].

⁷<https://docs.ray.io/en/latest/tune/index.html>

⁸<https://optuna.org/>

⁹<https://hyperopt.github.io/hyperopt/>

5

Implementation

This chapter explains the implementation process. Section 5.1 shows the amount of fine-tuning data which was gathered and where it comes from. The subsequent section 5.2 guides through the preprocessing of the data by splitting the applied preprocessing pipeline. The last section 5.3 shows the actually selected LLM and the corresponding fine-tuning approach applied.

5.1 Data Gathering

Source	No. of contracts	Loading Time
Etherscan	41,040 (~40.96 %)	16 h 58 min (~77.41 %)
GitHub	1,045 (~1.04 %)	4 h 4 min (~13.55 %)
DISL	58,102 (~57.99 %)	53 min (~4.03 %)
Total	100,187	21 h 55 min

Table 5.1: Number of contracts and loading time for each source. Etherscan accounts for almost 80 percent of the loading time, although only just over 40 percent of contracts originate from it.

All data was obtained from GitHub, Etherscan and the DISL Dataset. API calls were used for GitHub and Etherscan, which explains the significantly longer loading time, as these calls are limited to a certain number per second (a free API key was used for both cases). As some contracts from these sources were not written with Solidity but with Vyper, they have already been sorted out. The DISL dataset was obtained from Hugging Face. For all three sources, contracts with import statements were ignored due to the “Slither Analysis” step, which throws an exception if not all dependencies are resolved. Overall

100,187 contracts with a high diversity were gathered. Many kinds of typical Solidity applications mentioned in section 2.1 are represented, such as fungible tokens (e.g. ERC20), NFTs (e.g. ERC721), DeFi, DAOs, voting contracts, gaming contracts, etc. Table 5.1 shows the exact allocation of contracts and loading time to the respective sources.

5.2 Data Preprocessing

A preprocessing pipeline was created to bring the gathered data in an appropriate format for the subsequent LLM fine-tuning. The pipeline consists of seven chronological steps, namely:

- **Cleanup:** Removes unnecessary parts such as comments or blank lines from each Solidity file.
- **Formatting:** Converts the code in each file so that the final model only generates code in a correct format.
- **Slither Analysis:** Checks for vulnerabilities in each Solidity file of the dataset. The files are sorted by the vulnerabilities they contain, and vulnerability annotations are added to the line or construct in which the vulnerability was detected.
- **Splitting:** Splits each Solidity file according to the definitions (i.e. contract, interface or library) it contains. The required imports are added to the splitted files accordingly.
- **Similarity Check:** Checks for duplicate and very similar contracts and removes them accordingly.
- **Solhint Fixes:** Fixes some of the best practice issues detected by Solhint in the Solidity files.
- **Token Insertion:** Inserts special tokens into each remaining Solidity file that mark the end of a sequence, secure code or FIM code.

It also contains three optional steps, namely:

- **Vulnerability Repair:** Repairs some of the vulnerabilities detected during the “Slither Analysis” step to get more secure contracts.
- **Constructs Extraction:** Extracts constructs such as functions, modifiers or enums from each file.
- **Single-line Conversion:** Converts the code of each file into one line.

Each step is explained in more detail in the following subsections. For a detailed overview see figure 5.1. Listing 5.1 shows a short example of Solidity code before it was preprocessed. As one can see, there are many comments and blank lines (for reasons of space, some blank lines have been removed for this example).

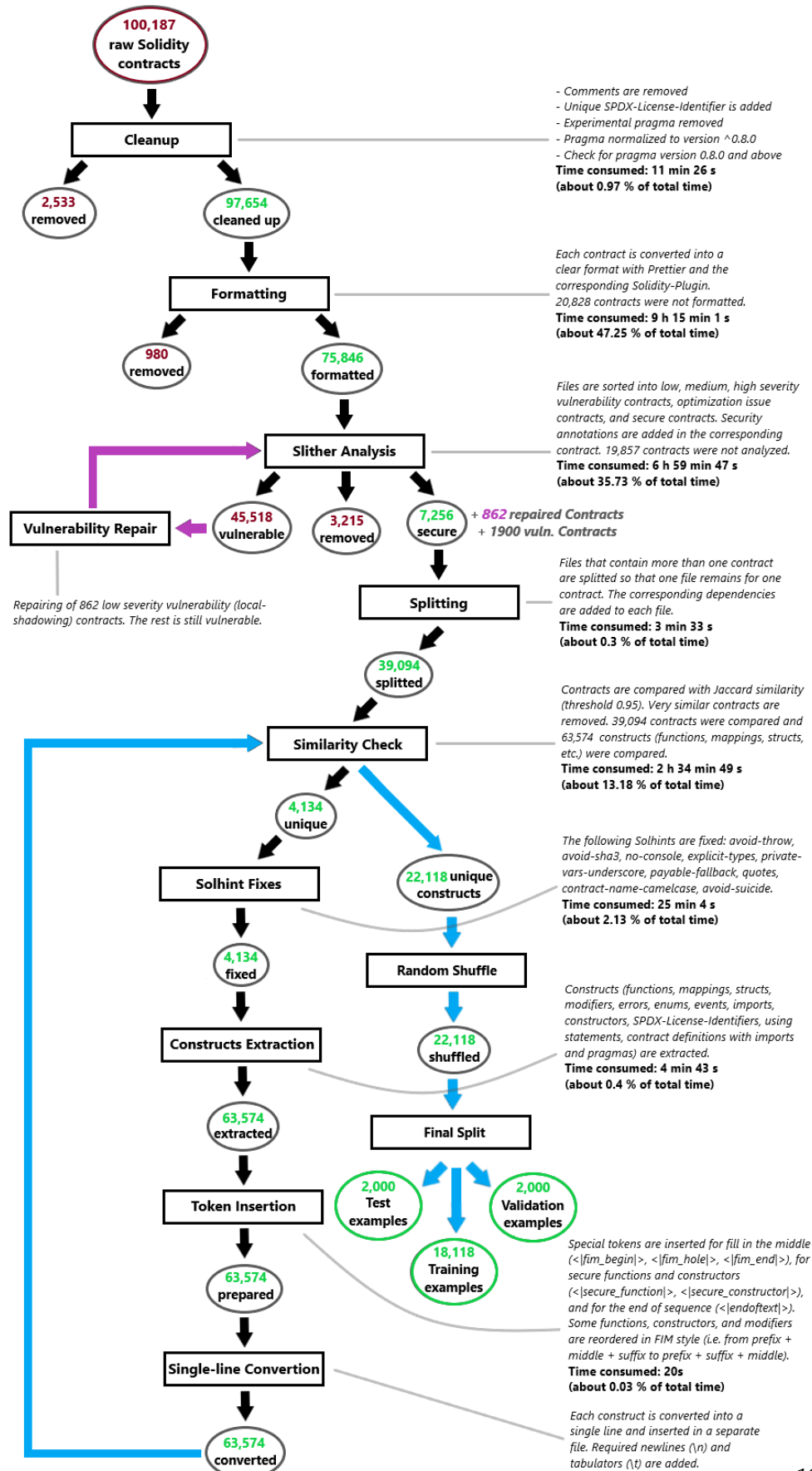


Figure 5.1: Visualization of the preprocessing pipeline. On the right-hand side is a brief description of the individual steps and the time required for that step.

5 Implementation

```
1 //SPDX-License-Identifier: MIT
2
3 // File: @openzeppelin/contracts/utils/math/SignedMath.sol
4
5
6
7 // OpenZeppelin Contracts (last updated v5.0.0) (utils/math/...
8
9
10 pragma solidity ^0.8.20;
11
12 /**
13  * @dev Standard signed math utilities missing in the Solidity ...
14  */
15
16 library SignedMath {
17
18     /**
19      * @dev Returns the largest of two signed numbers.
20      */
21
22     function max(int256 a, int256 b) ... returns (int256) {
23
24         return a > b ? a : b;
25
26     }
27
28     ...
29 }
30
31
32 }
```

Listing 5.1: Solidity code before any preprocessing

Cleanup

The pipeline starts with the “Cleanup” step. Unnecessary lines, such as single-line comments, multi-line comments and blank lines, were removed. The pragma version was normalized to version ^0.8.0 and experimental pragma versions as well as multiple pragma versions were removed. Every contract with a pragma version below 0.8.0 was sorted out. So the final model will only be able to generate Solidity code with techniques of version 0.8.0 and above. This is not a major limitation, as it is generally recommended to use the latest version due to security concerns. The SPDX license identifier was converted into a unique format, i.e. missing spaces were inserted and misspelled identifiers were replaced by the correct one. Listing 5.2 shows the changes (shown in bold) compared to listing 5.1. From the 100,187 gathered contracts, 97,654 survived this step. 2,533 were removed due to wrong pragma versions.

```

1 // SPDX-License-Identifier: MIT
2
3 pragma solidity ^0.8.0;
4
5 ... 19 newlines
6
7 library SignedMath {
8
9 ... 7 newlines
10
11     function max(int256 a, int256 b) ... returns (int256) {
12
13         return a > b ? a : b;
14
15     }
16
17 ...
18 }

```

Listing 5.2: Solidity code after cleanup step

Formatting

In the second step the contracts were formatted with the code formatter Prettier and the corresponding Solidity-plugin¹. This is necessary as otherwise the model will generate code in an incorrect format. Overall 75,846 contracts were formatted, 20,828 were not formatted (due to time constraints) and 980 contracts were removed due to errors inside the code Prettier could not handle. Unfortunately, this step was very time consuming. It accounts for almost half of the time required for the entire preprocessing. A multithreading approach was used to speed up the process. The formatting was done on an Intel i7-4790 CPU which worked under full load due to the multithreading.

```

1 // SPDX-License-Identifier: MIT
2
3 pragma solidity ^0.8.0;
4
5 library SignedMath {
6     function max(int256 a, int256 b) ... returns (int256) {
7         return a > b ? a : b;
8     }
9
10 ...
11 }

```

Listing 5.3: Solidity code after formatting step

¹<https://github.com/prettier-solidity/prettier-plugin-solidity>

Slither Analysis

Since the final model should only generate secure and optimized code (according to gas consumption), a Slither analysis was carried out for each file. Slither is a static analysis framework tailored for Solidity and Vyper code. It is able to detect 67 different vulnerability types, 21 informational issues (excluded from analysis, an example is *naming convention*) and 5 gas consumption issues. Unfortunately, it is only able to detect and not repair these issues. In general there exists no automatic repair tool for all mentioned issues. Hence, it was used to make annotations inside the code with comments and to sort all contracts by the severity of the vulnerability into high, medium and low vulnerability contracts, gas consumption issue contracts and secure contracts. The annotations should help the model distinguish between secure and vulnerable contracts/constructs and suggest, with comments, that the specific code is vulnerable.

```
1  /**
2   * WARNING Optimization Issue (immutable-states | ID: 21fb30b):
3   * DYORNFA._feeAddrWallet1 should be immutable
4   *
5   * Recommendation for 21fb30b: Add the 'immutable' attribute to
6   * state variables that never change or are set only in the
7   * constructor.
8   */
9  address payable private _feeAddrWallet1;
10
11 /**
12  * WARNING Vulnerability (unused-return | severity: Medium |
13  * ID: 3032186):
14  * AlcoholicApe.createUniswapPair() ignores      return value by
15  * IERC20(_pair).approve(address(_uniswap),type()(uint256).max)
16  *
17  * Recommendation for 3032186: Ensure that all the return values
18  * of the function calls are used.
19  */
20 function createUniswapPair() external onlyOwner {
21     require(!_canTrade, "Trading is already open");
22     _approve(address(this), address(_uniswap), _tTotal);
23     _pair = IUniswapV2Factory(_uniswap.factory()).createPair(
24         address(this),
25         _uniswap.WETH()
26     );
27     // unused-return | ID: 3032186
28     IERC20(_pair).approve(address(_uniswap), type(uint).max);
29 }
```

Listing 5.4: Security and optimization annotations after Slither analysis

To insert annotations at the right position in each file, a detailed understanding of the JSON output file produced by Slither was necessary. Any helpful information was extracted from these JSON files such as the type of vulnerability,

the exact line where it was detected, the construct (function, constructor, contract) in which it was detected, the severity of it, a short description of it, and the corresponding ID. The extracted severities were helpful for sorting them. If different severities have been extracted in one file, the file is sorted according to the highest severity. The exact line, as well as the construct in which it was discovered, was helpful in inserting annotations at these positions. At the exact line a short comment with the vulnerability type and ID was inserted. At the beginning of the parent construct of these vulnerabilities a detailed comment with the type, severity, id, description and recommendation how to avoid it, was inserted. The ID should help the model establish a connection between the description and the exact position of the vulnerability. The recommendation could not be extracted from the JSON file, hence a dictionary was created with all 67 vulnerability types, all 5 optimization issues and the corresponding recommendation how to avoid it as well as a short description of it. Both were obtained from the Slither GitHub page. Listing 5.4 shows an example for an unused-return vulnerability annotation and for an immutable-states optimization issue annotation. It is to mention that for reasons of space, multiline comments are used here. In the actual cases, single-line comments are used for the description and recommendation.

Type	No. of contracts	Repaired
Secure	4,386 (~7.83 %)	-
Opt. Issue	2,870 (~5.13 %)	1,815
High Severity	36,671 (~65.5 %)	-
Medium Severity	3,737 (~6.67 %)	-
Low Severity	5,110 (~9.13 %)	862
Removed	3,215 (~5.74 %)	-
Total	55,989	2,677

Table 5.2: Result of the Slither Analysis step. Almost two thirds of all contracts analyzed contain at least one high severity vulnerability.

A total of 55,989 contracts were analyzed and annotated in this way. Only 7,256 contracts had no security issues, but of these 2,870 had optimization issues. To increase the number of secure and optimized contracts 2,000 contracts with optimization issues were repaired. This was simple, as in most cases the issues consist of a missing constant or immutable keyword for a contract variable. It worked for 1,815 contracts, which no longer had any issues after the re-analysis. 3,215 contracts were removed during the analysis due to errors inside the code Slither could not handle. Due to time constraints 19,857 contracts were not analyzed. 45,518 contracts were vulnerable of which 5,110 had only low severity vulnerabilities, 3,737 had at least one medium severity vulnerability and 36,671 had at least one high severity vulnerability. To increase the number of secure contracts, some of the 5,110 low severity

contracts were repaired. Many of these were so-called *local-shadowing* vulnerabilities. Local shadowing means that a local variable of a function shadows a global variable of the contract due to an identical identifier. Hence, the local variable simply needs to be renamed. This was carried out for 2,550 low severity contracts, of which 862 were secure after re-analysis. This step is shown in figure 5.1 as optional step “Vulnerability Repair”. At the end of the analysis the dataset consisted of

- 6,201 secure contracts and
- 1,055 optimization issue contracts and
- 862 repaired contracts and
- 1,900 annotated vulnerable contracts.

This results in a total number of 10,018 contracts for further preprocessing. The analysis was partly done on Google Colab with a TPU consisting of 96 cores. The rest was analyzed with an Intel i7-4790 CPU. Everything is summarized in table 5.2.

Splitting

As in most cases a file contains more than one contract as well as libraries, interfaces and abstract contracts, the files are split up according to the various definitions. There are two reasons for splitting: firstly, it is best practice to have only one definition per file, and secondly, it helps to remove duplicates, which is explained in the next section. For each new file the SPDX license identifier and pragma version of the original one is added. To remedy the dependency problems that have arisen, the required imports are inserted. To achieve this, the splitting function searches for typical dependency patterns such as `contract .. is ..`, `interface .. is ..` or `using .. for ...`. For example, for the

- `contract Ownable is Context` an import of the abstract contract `Context`
- `interface IERC20Metadata is IERC20` an import of the interface `IERC20Metadata`
- `using SafeMath for uint256` an import of the library `SafeMath`

has to be inserted. But that are not all the dependencies that could arise. There could also be various instantiations inside the contract body such as `IUniswapV2Router02 uniswapV2Router ...`. Fortunately, after the previous step “Slither Analysis” there are only files left, which have all required dependencies inside that file, i.e. there are no unresolved dependencies. This is due to the fact that Slither cannot handle such files and they have therefore been removed. So it was sufficient to create a list with all the names of the various definitions for each file and then to search for this names in each definitions body. If there is a hit, such as with the instantiation described before, an import is added. For common imports (e.g. `IERC20`) a dictionary

was created with the name (IERC20) as key and the corresponding link (@openzeppelin/contracts/interfaces/IERC20.sol) as value. If a hit is inside this dictionary the link to the file is added as import statement, otherwise only the relative path to the file is added. Listing 5.5 shows a short example of a split contract. The dependencies that the splitting function must find and whose import statements it must insert are shown in bold.

```

1  // SPDX-License-Identifier: MIT
2
3  pragma solidity ^0.8.0;
4
5  import "./Platforme.sol" as Platforme;
6  import "@openzeppelin/contracts/interfaces/IERC20.sol" as IERC20;
7  import "@openzeppelin/contracts/access/Ownable.sol" as Ownable;
8  import "./UniswapRouterV2.sol" as UniswapRouterV2;
9  import "@openzeppelin/contracts/utils/math/SafeMath.sol" as SafeMath;
10 import "./IUniswapRouterV2.sol" as IUniswapRouterV2;
11
12 contract CREEPE is Platforme, IERC20, Ownable {
13     using SafeMath for uint256;
14     ...
15     UniswapRouterV2 private immutable Router2Instance;
16     ...
17     uint256 balance = IUniswapRouterV2.swap99(
18         ...
19     );
20     ...
21 }

```

Listing 5.5: Solidity code after splitting (dependencies are shown in bold)

Similarity Check

During this step duplicates or very similar contracts were removed. Every contract is compared to each other, which is done with Jaccard similarity. The Jaccard similarity is defined as the intersection of two sets divided by the union of the same two sets. Therefore any contract is tokenized, i.e. is now a set of keywords, identifiers, brackets, etc. The length of the intersection of two tokenized contracts is divided by the length of the union of these sets. If this value is above a given threshold one of the two contracts is marked as duplicate and removed. Now one can easily see why splitting helps to remove duplicates. Files are split up according to their definitions. In other words there are now many files with common case libraries (e.g. SafeMath), interfaces (e.g. IERC20) and (abstract) contracts (e.g. Context). Many of these are now removed, which would not necessarily be the case if splitting was not done. There could be a definition inside the file which leads to a sufficient difference of the tokenized sets to not exceed the threshold. It should also be mentioned here that the comments inserted during the Slither analysis were

removed for the similarity check. This speeds things up. The comparison of 39,094 contracts was done with a threshold of 0.95. After this step, only 4,134 contracts remained.

Solhint Fixes

The model should also be able to generate best practice code. To achieve this to some extent, the Solhint linter was used. Solhint is able to detect 16 best practice issues, 18 style guide issues, 10 gas consumption issues and 17 security issues. As one can see, Solhint is not nearly as powerful as Slither, but it has the advantage that it is able to fix some of the issues. These are the three security issues *avoid-throw*, *avoid-sha3* and *avoid-suicide*. The four best practice issues *explicit-types* (e.g. replaces `uint` by `uint256`), *quotes* (uses double quotation marks for all strings), *no-console* (removes `console.log` statements such as `console.log('test')`) and *payable-fallback*. As well as the two style guide issues *private-vars-underscore* (adds a single underscore for non-external functions and state variables) and *contract-name-camelcase* (forces to use camelcase style for contract names). Unfortunately, Solhint is not even close to fix all issues. This means that the final model will only generate best practice code to some extent. To avoid this one could write a script to fix much more of the known issues, but this would be very difficult and time consuming and therefore go beyond of the scope of this work. Another opportunity would be to gather only Solidity smart contracts that are already written in best practice manner. But even this is very time consuming and difficult, if not impracticable, if enough contracts have to be collected to fine-tune an LLM. A third way would be to synthetically create contracts with best practice code, but then there would be a lack of diversity. The fixing did not remove contracts.

Special Token Insertion

From this step onwards, various final datasets were created with different special token approaches. For example, in one case special tokens have been added to help the model to distinguish between different constructs within a contract or between different contract types and to know exactly where a particular construct ends. So, for example, the token `[BEG_FUN]` has been inserted at the beginning of functions and the token `[END_FUN]` at the end. This has now been done for each construct (i.e. contracts, libraries, functions, interfaces, events, etc.). For vulnerable constructs special tokens such as `[VUL_BEG_FUN]` and `[VUL_END_FUN]` were inserted. This was done with the help of the security annotations that were inserted during the “Slither Analysis” step and should further help the model to distinguish between secure and vulnerable constructs. Another approach was to only insert end tokens or to extract each construct into a single file and then insert special tokens. Different names were also used for the special tokens. After various tests, it was decided to use the following approach.

First of all, all relevant constructs were extracted (i.e. functions, modifiers, constructors, errors, enums, events, structs, imports, mappings, SPDX-License-Identifiers, using statements and contract definitions with imports and pragmas). Then the FIM approach explained in section 2.4 was applied to all functions, modifiers and constructors with more than two lines (blank lines are not included) within the respective constructs body. An upper limit for the lines is also applied, which depends on the respective construct. For this project a value of 0.4 was chosen for the FIM ratio, i.e. 40 % randomly selected lines were moved to the end and the PSM transformation was applied. Different FIM ratios were tried such as 0.35 or 0.5. On the surface, the FIM ratio of 0.4 delivered the best splits, but this could definitely be investigated further. The inserted special FIM tokens are `<|fim_begin|>`, `<|fim_hole|>` (the placeholder for the missing part) and `<|fim_end|>`.

The approach was also applied to imports. For all contracts with imports, the code from the pragma definition to the contract definition was extracted. The imports between these two are the middle part and moved to the end (no FIM ratio was applied here, all imports were always moved to the end). In other words, if a developer writes a contract definition such as `contract Example is ERC20, Ownable {`, the model should recognize the dependencies and fill in the corresponding imports.

Additionally, for all secure functions and constructors a special token was inserted at the beginning, namely `<|secure_function|>` and `<|secure_constructor|>`. These tokens force the model to generate only secure versions of these constructs during inference. The FIM approach was also applied for vulnerable functions and constructors, which means that the model learns to fill in security annotations for vulnerable lines. In other words, if the developer writes a vulnerable line, the model should add a security annotation in form of a comment. This would be the name of the vulnerability given by Slither, as described in the “Slither Analysis” step.

Finally, a special token was inserted for the end of the sequence, namely `<|end_of_sequence|>`. This is necessary so that the model knows where to end the code generation. The special tokens must be added as additional tokens to the tokenizer used to tokenize the dataset. This is necessary as otherwise the tokenizer might split the special tokens (e.g. `<|secure_function|>` into `<,|,secure,_,function,|,>`).

Each construct was then converted into a single line (the required newlines and tabulators are added with `\n` and `\t`) and inserted in a separate file. After applying these steps, referred to in figure 5.1 as “Constructs Extraction”, “Token Insertion” and “Single-line Conversion”, 63,574 converted constructs were obtained for the 4,134 contracts obtained after the “Solhint fixes” step. Similar constructs are removed with a Jaccard threshold of 0.95 for the same reasons already explained in the “Similarity Check” step. There are now 22,118 constructs left. These are shuffled and finally split into 2,000 examples for validation, 2,000 examples for testing and 18,118 examples for training.

Listing 5.6 shows extracted functions with inserted special tokens (except for the `<|end_of_sequence|>` token). Table 5.3 shows the number of individual constructs in the final datasets. It also shows how many of the total constructs were FIM transformed (about 55.55 % of the total dataset) and how many are vulnerable (about 10.15 % of the total dataset). The final dataset can be found on Hugging Face under `fbnhnsl/FIM_Solidity_Dataset`. The code of the preprocessing pipeline can be found on GitHub under `FabianHensel/Solidity_AI-powered_Automatic_Code_Completion_Tool`.

```

1 // imports after FIM transformation
2 <|fim_begin|> pragma solidity ^0.8.0;
3
4 <|fim_hole|>
5
6 contract Example is Ownable, IERC20 { <|fim_end|>
7 import "@openzeppelin/contracts/access/Ownable.sol" as Ownable;
8 import "@openzeppelin/contracts/interfaces/IERC20.sol" as IERC20;
9 -----
10 // secure function after FIM transformation
11 <|secure_function|> <|fim_begin|> function excludeFromFees ... {
12 <|fim_hole|>
13     emit ExcludeFromFees(account, excluded);
14 } <|fim_end|> _isExcludedFromFees[account] = excluded;
15 -----
16 // secure function without FIM transformation
17 <|secure_function|> function setEarlySellTax(bool onoff) ... {
18     enableEarlySellTax = onoff;
19 }
20 -----
21 // vulnerable function with FIM transformation
22 <|fim_begin|> function updateMarketingWallet(...) ... {
23     emit marketingWalletUpdated(newMarketingWallet, ...);
24 <|fim_hole|>
25     marketingWallet = newMarketingWallet;
26 } <|fim_end|> // missing-zero-check vulnerability

```

Listing 5.6: Extracted Solidity constructs with inserted special tokens and FIM transformations

Construct	Training	Validation	Test	Total
Function	10,508	1,131	1,150	12,789
<i>Vuln. Function</i>	1,701	164	205	2,070
<i>FIM Function</i>	6,161	664	674	7,499
<i>Vuln. FIM Function</i>	1,465	135	183	1,783
Constructor	1,877	240	222	2,339
<i>Vuln. Constructor</i>	138	24	13	175
<i>FIM Constructor</i>	1,146	163	137	1,446
<i>Vuln. FIM Constructor</i>	126	21	11	158
Modifier	524	64	55	643
<i>FIM Modifier</i>	524	64	55	643
Import	2,807	325	308	3,440
<i>FIM Import</i>	2,198	259	241	2,698
Struct	231	20	24	275
Event	847	86	95	1,028
Error	306	26	35	367
Enum	28	3	2	33
Mapping	922	101	100	1,123
Using	34	4	9	47
SPDX-Identifier	34	0	0	34
Total	18,118	2,000	2,000	22,118

Table 5.3: Composition of the final dataset. FIM transformed constructs are shown in yellow, vulnerable constructs are shown in red, and the constructs to which both apply are shown in pink. A few things stand out in particular: no SPDX-Identifier made it into the test and validation set, very few using statements and enums are present overall, the FIM transformation was applied to all available modifiers, and functions are present disproportionately often.

5.3 Model Fine-tuning

The fine-tuning was carried out on Google Colab using an Nvidia L4² Tensor Core GPU available there. The L4 has a GPU memory of 24 GB, which enables the fine-tuning of an LLM with PEFT methods and the execution of inference of the model. The chosen LLM for fine-tuning was the Deepseek-Coder-1.3b-base³ model from DeepSeek. It is worth mentioning here that other code LLMs were also fine-tuned, such as Starcoder2-3b⁴, CodeLlama-7b-hf⁵, and Starcoderbase-1b⁶, for all of which good results were obtained. The final decision to use DeepSeek's model was made due to its relatively small size of 1.3 billion parameters, which is good as the final training dataset is also quite small. If a large model is used for a small dataset, the effect of fine-tuning on the model weights may not be as significant as for a smaller one. Other obvious reasons for using such a small model were the training duration and storage requirements. Another important reason was that it was already pretrained with Solidity code, which means that it is easier for it to handle Solidity constructs. This was not entirely clear for the other code LLMs, as it is not specified anywhere. Deepseek-Coder-1.3b-base is a CLM. It is the smallest model of the Deepseek-Coder series (other sizes are 5.7 billion parameters, 6.7 billion parameters or 33 billion parameters). The pretraining was done on 1 trillion tokens of various programming languages.

The first step for the fine-tuning was to load the preprocessed dataset. This was done with the Hugging Face library `datasets`. The dataset was then tokenized with the same tokenizer used for the pretraining of Deepseek-Coder-1.3b-base. The tokenizer is based on BPE and was obtained with the Hugging Face library `transformers`. BPE was originally developed to compress text and was first used for tokenization in OpenAI's GPT models. It iteratively merges the most frequent adjacent symbol pairs in code (or text) into new tokens, balancing token granularity and vocabulary size for efficient code representation. The original vocabulary size of the model is 32,022 and with special tokens 32,028. It was important to resize the models embeddings due to the additional tokens. If a symbol occurs which is not in the vocabulary, an unknown token was used to tokenize it. The input sequence length for the model was chosen to be 256 tokens. This was a good compromise between the duration of fine-tuning and the generation capability. Longer input sequences getting truncated and shorter input sequences getting padded with a special [PAD] token.

After the tokenization of the dataset each example consists of `input_ids` and an `attention_mask`. An `input_id` is an unique integer ID from the vocabulary of the model to which each token is mapped, i.e. a Solidity con-

²<https://www.nvidia.com/en-us/data-center/l4/>

³<https://huggingface.co/deepseek-ai/deepseek-coder-1.3b-base>

⁴<https://huggingface.co/bigcode/starcoder2-3b>

⁵<https://huggingface.co/codellama/CodeLlama-7b-hf>

⁶<https://huggingface.co/bigcode/starcoderbase-1b>

struct is now a list of numbers. They are not to be confused with the token embeddings. They have no semantic meaning and are passed through an embedding layer to obtain the high-dimensional numerical vectors. The `attention_mask` is a list of 0s and 1s and is necessary as transformers operate on fixed-length sequences, but the inputs are of varying length. As already mentioned, shorter input sequences are padded to match the maximum sequence length of 256 tokens. The `attention_mask` now helps the model to ignore these tokens during computation by associating relevant tokens with 1s and non-relevant tokens (padding tokens) with 0s. The labels, i.e. the target tokens that the model should predict, used by the loss function to compute the difference between ground truth and the model's predictions, must then be added for each example, which is easy for CLMs as only a direct copy of the `input_ids` needs to be added. Its worth mentioning, that also a special FIM data collator was created. Typically, the `DataCollatorForLanguageModeling` class from the `transformers` library is used for language modeling tasks. It dynamically pads the inputs to the maximum sequence length. The FIM data collator serves a different purpose. It assigns the special value -100 to all labels in the prefix and suffix part of an FIM example. The labels in the middle part are kept as-is. This special value indicates that a particular token should be ignored when computing the loss. This makes sense because the prefix and suffix part should not be predicted, only the middle part. In other words, the model is only penalized for incorrect predictions of the middle part. Obviously, it makes no sense to apply this assignment of special values to the (prefix and suffix part) `input_ids`, as the model cannot then process the entire context. For non-FIM examples, nothing changes for the labels.

A QLoRA approach was chosen for the fine-tuning. As already mentioned in section 4 it allows for resource efficient fine-tuning on a single GPU. QLoRA consists of two steps, namely quantization and LoRA. Quantization is the process of mapping continuous numerical values to a finite set of discrete values. Every computer performs quantization by default, since a continuous value is represented in a finite-precision format such as FP16 or FP32. This also means that it makes sense to describe quantization in the context of QLoRA as the process of converting an input from a high-precision representation to one with lower precision. LLMs are pretrained in FP32 or mixed-precision format, which computes the parameter updates (i.e. the forward and backward computations) in FP16 or bfloat16 (a special datatype invented by Google for deep learning, which speeds up the computations) and stores the parameters in FP32. In QLoRA, the stored parameters of the pretrained model are converted into a 4-bit or 8-bit format (float or integer), but the parameter updates are still computed in FP16 or bfloat16, for which the parameters must be dequantized. The implementation of quantization was done with the `BitsAndBytesConfig` class from the `transformers` library. For this project the model parameters were converted into a 4-bit normal float format (a special datatype developed by the QLoRA team), dou-

ble quantization was used (i.e. the quantization constants are also quantized, which saves additional memory space) and the `bfloat16` datatype was chosen for the model to compute the parameter updates. Now, the model was loaded with the `AutoModelForCausalLM.from_pretrained` function from the `transformers` library and the described config. The model was also loaded with flash attention 2 [Dao24], which is an optimized algorithm that accelerates the attention computations in LLMs while minimizing memory usage. LoRA decomposes the weight update matrix into two trainable low-rank matrices. The original weights of the pretrained model remain frozen while these two matrices are updated during training. In other words, only a small set of the original weights was updated.

Four components must be specified for the implementation of LoRA, namely rank, alpha, dropout, and target modules. The rank is used to determine the size of the matrices. A higher rank means that more weights have to be trained, which leads to higher memory consumption and a lower training speed, but to a higher learning capacity. The alpha value is a scaling factor for the weight changes. A higher value means a greater influence of the weight changes on the original model weights after they have been added back. LoRA dropout is a regularization technique used to prevent overfitting by randomly setting a fraction of the elements in the trainable low-rank matrices to zero during training. The dropout value represents the probability that an element in these matrices will be dropped (i.e. a weight is set to zero). The target modules are the layers of the models network where LoRA is applied. For transformer-based LLMs, these are usually the attention layers, as they contain the majority of the trainable parameters and significantly influence the performance of the model. The implementation of LoRA was done with the `LoraConfig` class and `get_peft_model` function from the Hugging Face library `peft`.

After the model was prepared for QLoRA fine-tuning, the fine-tuning arguments were defined with the `SFTConfig` class from the Hugging Face library `trl` (abbreviation for transformer reinforcement learning). These arguments are learning rate, weight decay, batch size, gradient accumulation steps, warmup steps, training epochs, optimizer, gradient checkpointing, and packing. The learning rate and optimizer were already explained in 2.3. But one important fact about the learning rate should also be mentioned here. If the learning rate is too high, the minimum of the loss function may be overshoot, whereas if the learning rate is too low, the minimum may never be reached. In other words, for both cases, the model has trouble in achieving a high level of accuracy. The learning rate must therefore be chosen carefully. The chosen optimizer was paged AdamW 8-bit [LH19], which is a memory efficient variant of the AdamW optimizer. It quantizes the state values, i.e. the values which the optimizer keeps track of during training, from FP32 or FP16 into an 8-bit format. "Paged" refers to the ability of the optimizer to perform dynamic memory management such as moving parts of the optimizer state to the CPU memory if the GPU runs out of memory [Mar24]. Like dropout or batch normalization, weight decay is a regularization technique used to

prevent overfitting. The batch size determines the number of examples of the training dataset that are processed in a single forward pass before the gradients are computed and the model parameters are updated during the backward pass. In other words, a larger batch size means that the model sees more examples before it performs a parameter update. This leads also to faster training, since less parameter updates are performed in one epoch, but it requires more GPU memory. An epoch is defined as one complete forward pass through the entire training dataset during training, i.e. the model has seen any example. For this project the batch size was set to eight. To speed up the training nevertheless, gradient accumulation was used, which “simulates” a larger batch size. Rather than updating the model weights after each batch, the gradients are accumulated over a specified number of batches, and the optimizer updates the weights only after this number of batches has been processed. During warmup the learning rate is set to a very small value and increases linearly over the warmup steps until it reaches the base learning rate. This helps to reduce the risk of unstable updates and improves the overall stability of the training. The maximum training steps define the number of batches after which the training will be terminated. For example, one epoch for the 18,118 training examples, a batch size of eight, and four accumulation steps would consist of $18,118 / (8 \times 4) \approx 566$ maximum steps. Gradient checkpointing [CXZG16] is used to further reduce the required memory.

To determine the best possible values for the hyperparameters of the model a hyperparameter optimization approach was applied. It was implemented using the `Ray Tune` library. The search space consists of learning rate, weight decay, gradient accumulation steps, warmup steps, and LoRA rank, alpha, and dropout. The applied search algorithm was Optuna search from the Optuna library. ASHA was used as scheduler. 14 different configurations were tested for one epoch each. It took a total of 11 hours and 19 minutes. Table 5.4 shows the result of each trial. The best trial is marked in yellow. It has a validation loss of about 0.84 and a perplexity of about 2.32 (on the validation dataset). Figure 5.2 shows the graphs of the validation loss and figure 5.3 the graphs of the training loss of some trials (they were obtained with `Weights & Biases`⁷, a useful tool for tracking a run). The graphs have a different size on the x-axis due to the varying gradient accumulation steps (the batch size was set to four for each trial). For example, in configuration 10 an epoch consists of $18,118 / (4 \times 2) \approx 2,264$ steps, while in configuration 12 an epoch consists of $18,118 / (4 \times 4) \approx 1,132$ steps. Validation was done after about max steps divided by batch size. In other words, for config 10 the first validation was done after 566 steps and for config 12 the first validation was done after 283 steps. In this way, each configuration has always seen exactly the same number of examples during training before the validation was carried out. The best configuration of the hyperparameter optimization was used to fine-tune the model. This was done with the `SFTTrainer` class from

⁷<https://wandb.ai/site>

the `trl` library for two epochs. Only very slight improvements are achieved after two epochs. The fine-tuning took a total of 1 hour and 23 minutes. The training loss was about 0.7 and the validation loss about 0.75. Figure 5.4 shows the graph of the validation loss and figure 5.5 shows the graph of the training loss of the final model. The final model can be found on Hugging Face under `fbnhnsl/Finetuned_deepseek-coder-1.3b-base_Solidity_Constructs`. The code used for fine-tuning the model can be found on GitHub under `FabianHensel/Solidity_AI-powered_Automatic_Code_Completion_Tool`.

Trial	lr	ws	wd	as	l_r	l_a	l_d	d (s)	el	p
1	2.547e-05	100	0.0482	8	64	16	0.0875	2648.5	1.225	3.403
2	3.392e-05	150	0.0394	4	64	16	0.0689	2664.1	1.112	3.041
3	1.522e-05	100	0.0825	8	32	128	0.0389	2651.5	1.129	3.094
4	0.000208	150	0.0443	2	16	32	0.0458	2669.9	0.847	2.333
5	0.0001759	50	0.0516	4	8	32	0.0771	2610.3	0.873	2.393
6	0.0001086	50	0.0101	4	32	32	0.0853	2649.4	0.902	2.465
7	0.0002465	150	0.0162	4	32	16	0.0343	2667.5	0.878	2.406
8	0.000463	50	0.0682	8	16	16	0.0685	2645.9	0.872	2.391
9	5.244e-05	50	0.0286	2	32	128	0.0168	2625.6	0.867	2.379
10	2.487e-05	150	0.0662	2	32	16	0.0242	2619.2	1.083	2.954
11	0.0001599	100	0.0534	2	64	64	0.0935	2698.3	0.842	2.322
12	1.152e-05	150	0.0969	4	8	8	0.0411	2606.2	1.321	3.748
13	7.563e-05	150	0.0547	4	64	64	0.0898	2631.6	0.899	2.456
14	4.393e-05	100	0.0282	4	8	16	0.0437	2605.1	1.060	2.889

Table 5.4: Hyperparameter optimization results (lr = learning rate; ws = warmup steps; wd = weight decay; as = accumulation steps; l_r = lora rank; l_a = lora alpha; l_d = lora dropout; d = duration (s); el = eval loss; p = perplexity). The best trial is marked in yellow.

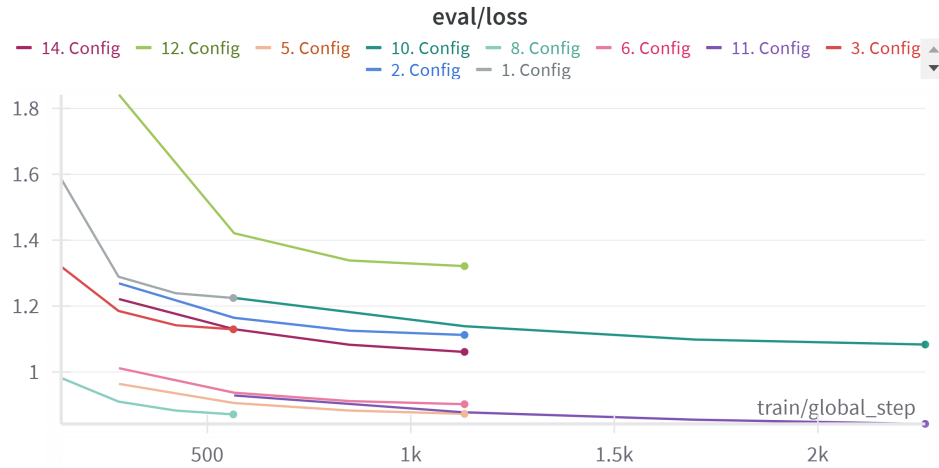


Figure 5.2: The graphs of the validation loss of some Ray Tune trials. The x-axis shows the training steps and the y-axis shows the validation loss. Configuration 11 has the lowest validation loss after one epoch.

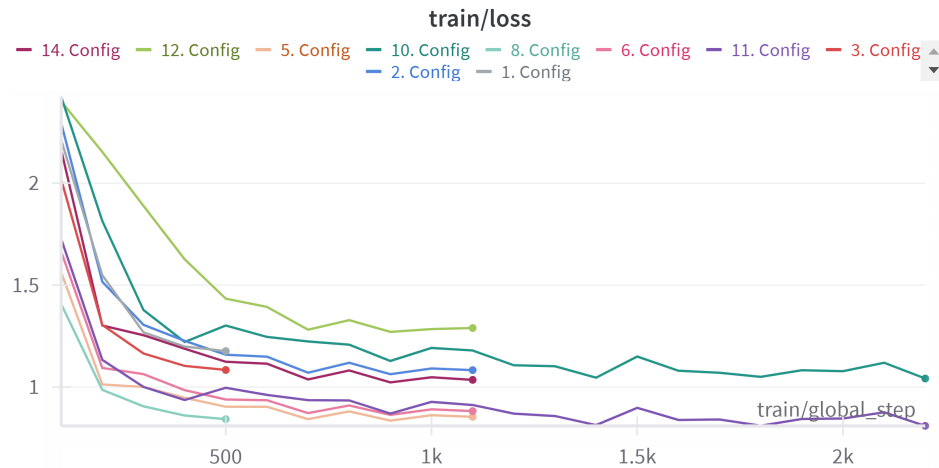


Figure 5.3: The graphs of the training loss of some Ray Tune trials. The x-axis shows the training steps and the y-axis shows the training loss. What is striking is that some graphs are exactly the same except for the shift in the y-axis.

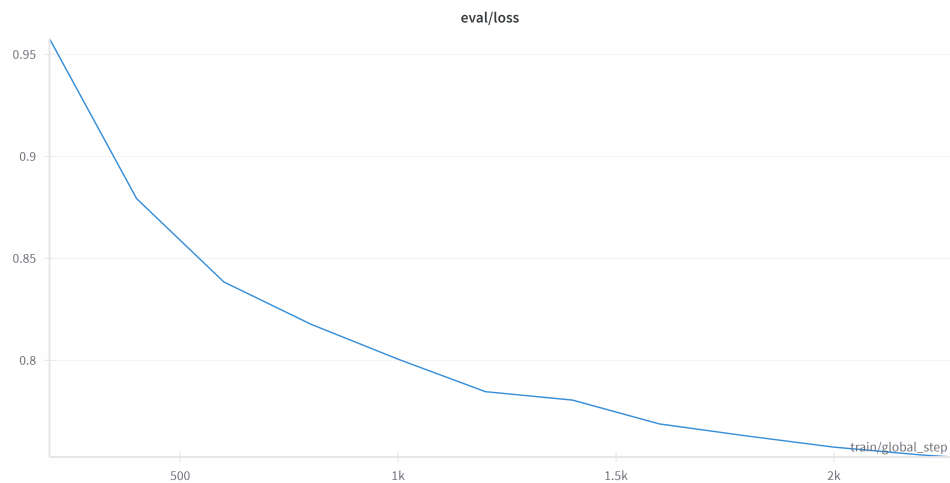


Figure 5.4: The graph of the validation loss for fine-tuning the model with the best configuration of the hyperparameter optimization for two epochs and batch size eight.

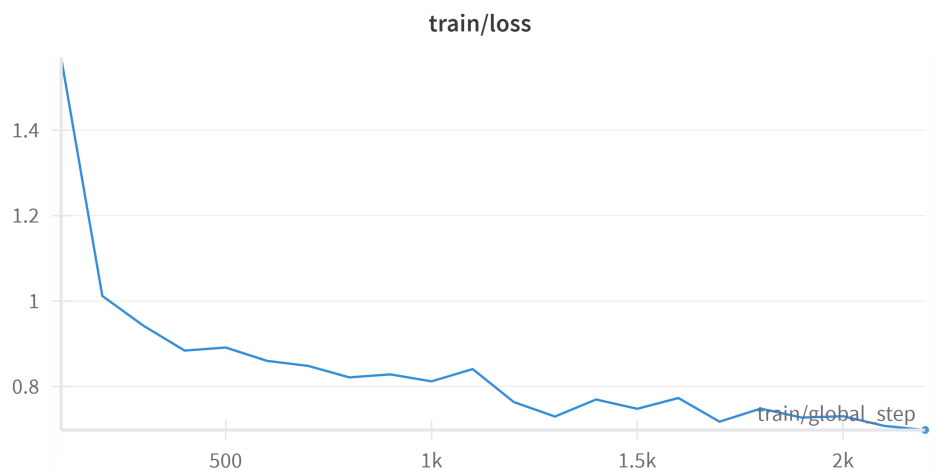


Figure 5.5: The graph of the training loss for fine-tuning the model with the best configuration of the hyperparameter optimization for two epochs and batch size eight.

6

Evaluation

This chapter presents the evaluation of the fine-tuned model. The model was evaluated in comparison to its base model (i.e. the pretrained model without fine-tuning). Five different tasks were evaluated and three evaluation metrics were used, namely perplexity, BLEU, and METEOR.

Perplexity is the exponential value of the cross-entropy loss, which measures how well a model's predicted distribution matches the true distribution of a sequence or token. A low perplexity means more reliable and accurate predictions. 1 would be a perfect perplexity, i.e. the next token is predicted with 100% certainty. A perplexity of 50,000 for a model with a vocabulary of 50,000 would mean that the model behaves in a random way, i.e. the vocabulary size is the upper limit for the maximum possible perplexity. A perplexity of 10 would mean that it reduces uncertainty to the level of choosing between 10 equally likely options for the next token in the sequence. The "goodness" of the perplexity depends on factors such as the size of the model, the size of the vocabulary and the specific task for which the model was trained. A larger model usually leads to lower perplexity (as it can capture more complex patterns), while a larger vocabulary leads to higher perplexity (as the model has more tokens to choose from). In addition, code LLMs achieve better scores than natural language LLMs due to the structured and repetitive nature of code. The perplexity was evaluated on the test dataset with the `evaluate` method from the `SFTTrainer` class. The fine-tuned model achieves a perplexity of 2.19, while the base model achieves a perplexity of 12.08. In other words, the fine-tuning has significantly improved the models prediction capabilities. However, perplexity says nothing about the lexical correctness of the code (or in other words, the correctness against a reference output). It simply indicates how uncertain the model is about the sequences it generates. Additional evaluation metrics were therefore applied.

BLEU is an n-gram matching algorithm, i.e. it measures the overlap of a

candidate output (what the model actually generates) and a reference output (what the model should ideally generate) by splitting both into n tokens and comparing these tokens. A score of 1 would indicate an exact overlap, while a score of 0 would mean a total mismatch. CodeBLEU extends this by comparing the code not only on a lexical level, as this says nothing about whether the generated code also corresponds to the syntactic and semantic rules of a particular programming language. Therefore, it additionally compares the syntactic structure with the corresponding abstract syntax trees and the data flow by extracting the data dependency graph of the candidate code and reference code. The final score is then calculated as the weighted sum of these three components. Unfortunately, there exists no CodeBLEU implementation for Solidity, and it is a non-trivial task to create such an implementation. Therefore, only BLEU was applied and in addition METEOR. The BLEU evaluation was done with the Hugging Face library `evaluate`.

METEOR also applies exact word matching, but in addition to that it can be used for structural and semantic matching. For example, it considers the code order (but an incorrect order is not penalized as much as it is with BLEU, which is especially useful if, for example, function parameters are in a wrong order), synonyms for identifiers (with the help of WordNet, a database of semantic and lexical relationships between English words), and code structure (missing spaces, tabulators and line breaks are penalized). The scoring system is based on precision, recall, and a penalty term. Precision measures the fraction of tokens in the candidate code that also appear in the reference code and recall measures the fraction of tokens from the reference code that also appear in the candidate code. They are combined to the so-called F-mean (a weighted harmonic mean), which is weighted towards recall. This weighting is essential for the evaluation of code completion, as it ensures that the generated code contains as much content as possible from the reference code (i.e. critical keywords, operators, and brackets). The penalty term is used to quantify the disruption in the code order. This is done by grouping consecutive matching tokens into “pieces”. There will be exactly one piece if all tokens of the generated code match with the tokens of the reference code in the right order. If matching tokens do not appear in the correct order, they form a smaller piece, resulting in a total of more than one piece. The penalty term is calculated by dividing the number of pieces by the total number of matching words, and the final METEOR score (a value between 0 and 1) is computed by multiplying the F-mean with the difference of 1 and the penalty term. In other words, many fragmented matches (i.e. matching tokens that are scattered or appear out of order) would lead to a worse score. It is important to note here that a high METEOR score in no way indicates that the code is functionally correct. Additional unit tests must be used to test this property. But it is a good indicator for lexical, syntactical, structural and, to a certain extent, semantical correctness of the code. Conversely, a low score does not necessarily mean that the code is incorrect, as the model may have generated code that is fundamentally different from the reference code but is still correct.

In repeated evaluations, however, a frequently resulting low score indicates incorrectly generated code. The METEOR evaluation was done with the `meteor_score` function from the `nltk` library, and the tokenization of the Solidity code required for this was carried out with the class `SolidityLexer` and the class `Token` from the `pygments` library.

The evaluation of infilling capabilities was done by randomly picking 10 FIM examples from the test dataset. These examples were divided into references (the middle part that the model should predict) and inputs (prefix and suffix part). For each of the inputs, predictions were generated. To achieve this, each input was encoded with the tokenizer, fed into the model to generate the outputs, and finally the outputs were decoded with the tokenizer. The predictions depend on two values, namely number of beams and temperature. The number of beams (a beam search is applied) determines the number of candidate sequences that are simultaneously explored and expanded at each step during the generation process. Instead of always choosing the single most probable token, the model considers alternative tokens to extend each candidate sequence, which allows for a broader search for optimal outputs. The temperature controls the randomness of the token selection during generation. A higher temperature value leads to more random generations, while a lower temperature leads to precise generations. The number of beams was set to 4 and the temperature to 0.3, as precise generations are required for code completion. But other values could be investigated further in any case.

For each of the 10 generated code sequences and reference sequences, the BLEU score and the METEOR score were computed, and the final score for both is the average of the 10 scores. The evaluation process was repeated 5 times for (vulnerable) FIM functions, modifiers, and imports. It took a total of 38 minutes. The results for each run and the average score of all runs can be seen in table 6.1. The fine-tuned model clearly outperforms the base model in both metrics in every run. The results also reflect exactly what was observed during manual tests. The generated code is structurally, semantically, syntactically and lexically correct in many cases. The model faces almost no trouble in generating imports given the contract definition and its dependencies. It also performs quite well on modifiers and secure functions. If the `<|secure_function|>` token was used for functions, it has almost never generated vulnerable code. The big difference between the BLEU score and the METEOR score for modifiers and functions is due to the fact that often only a few lines need to be generated for them (especially for modifiers). In such cases, if the model generates synonyms for identifiers or different strings in require statements (or other slight modifications), the BLEU score tends to be very low, while the METEOR score changes only slightly. Unfortunately, the model performs comparatively worse on vulnerable FIM functions. In about 4 of 10 cases it does not generate security comments. The main reason for this could be that the model has not seen enough vulnerable functions (and therefore security comments) during the training.

	1. Run	2. Run	3. Run	4. Run	5. Run	Avg. Score
FIM Functions						
PM BLEU	0.03	0.04	0.01	0.02	0.04	~0.03
FM BLEU	0.69	0.55	0.51	0.55	0.5	~0.56
PM METEOR	0.2	0.16	0.19	0.16	0.26	~0.19
FM METEOR	0.87	0.77	0.76	0.79	0.77	~0.79
Vuln. Hits	0	0	0	0	0	0
Vuln. FIM Functions						
PM BLEU	0	0.02	0	0	0	<0.01
FM BLEU	0.49	0.45	0.56	0.41	0.5	~0.48
PM METEOR	0.02	0.11	0.12	0.07	0.09	~0.08
FM METEOR	0.63	0.63	0.64	0.53	0.53	~0.59
No Comments	4	4	4	5	4	4.2
FIM Modifiers						
PM BLEU	0	0	0	0	0.01	<0.01
FM BLEU	0.45	0.32	0.23	0.56	0.49	~0.41
PM METEOR	0.12	0.1	0.2	0.13	0.14	~0.14
FM METEOR	0.82	0.67	0.76	0.87	0.79	~0.78
FIM Imports						
PM BLEU	0	0	0	0	0	0
FM BLEU	0.95	0.84	0.98	0.89	0.96	~0.92
PM METEOR	0.18	0.19	0.19	0.22	0.18	~0.19
FM METEOR	0.98	0.97	0.99	0.97	0.98	~0.98
Non-FIM Generation						
PM BLEU	0.11	-	-	-	-	0.11
FM BLEU	0.55	-	-	-	-	0.55
PM METEOR	0.3	-	-	-	-	0.3
FM METEOR	0.9	-	-	-	-	0.9
Vuln. Hits	0	-	-	-	-	0

Table 6.1: BLEU and METEOR evaluation results on the test dataset for five different tasks (PM = Pretrained Model; FM = Fine-tuned Model). Each task, with the exception of the generation of non-FIM code sequences, was evaluated in five runs, and the average of these was taken. “Vuln. Hits” means that the model has generated vulnerable code. “No Comments” means that no vulnerability annotations were generated where they should actually be.

The evaluation of general automatic code completion (i.e. non-FIM generation) was done by creating 20 partial inputs (consisting of function headers, modifier headers, events, etc.) and 20 corresponding reference completions. Here, too, the fine-tuned model performs significantly better than the base model. It has hardly any problem to generate code with the correct structure.

Functions were not completed with vulnerable code. But unfortunately this was sometimes the case in the manual tests. The difference between the two metric scores is due to the fact that the model often generates synonyms for identifiers and a different order for parameters.

Conclusion

This research investigated the fine-tuning of a code LLM to achieve automatic code completion for Solidity, the primary programming language used in smart contract development on the Ethereum blockchain. The objective was to enhance the model's ability to generate syntactically correct, contextually relevant, and efficient Solidity code while avoiding security vulnerabilities and incorporating coding best practices.

By leveraging a pre-trained transformer-based architecture and adapting it to a specialized Solidity dataset obtained with a comprehensive preprocessing pipeline, the model demonstrated significant improvements in code completion accuracy compared to the base model. Both in FIM code completion and general code completion. The integration of security-aware fine-tuning enabled the generation of more secure smart contract code by incorporating patterns that mitigate vulnerabilities or annotations that highlight vulnerabilities in the form of comments in the code. The evaluation results indicate that fine-tuning enhances the model's ability to produce intelligent code completions in Solidity development.

Despite these advancements, certain challenges remain. The model's ability to generalize across diverse Solidity codebases depends heavily on the quality and diversity of training data. For example, the model is not able to generate certain code techniques from Solidity versions below 0.8.0. Additionally, while security annotations improve the security of generated code, ensuring full compliance with evolving security standards in smart contract development requires ongoing updates and refinements. Another challenge is to convert each code example so that it complies with Solidity's best practices, as Solhint is only suitable for this to a limited extent.

Future research could explore the integration of the model in integrated development environments (IDEs) and optimizations in model efficiency to enhance usability. Expanding the dataset to include much more complex real-

world, secure and vulnerable (with annotations) code examples as well as edge cases could improve generalization and robustness. This applies in particular to the generation of security comments. No comprehensive evaluation of functional correctness was carried out in this work. This could be done with the aforementioned extrinsic evaluation metric Pass@k. Additionally, evaluating the model's performance in interactive settings, such as automated vulnerability detection, could further enhance its practical applications.

By advancing LLM fine-tuning for Solidity, this research contributes to the broader goal of improving the efficiency, accessibility, and security of smart contract development. AI-powered code completion has the potential to streamline blockchain programming, reduce human error, and foster a more secure and developer-friendly decentralized ecosystem.

Bibliography

- [Ala20] N. Alarcon: *OpenAI Presents GPT-3, a 175 Billion Parameters Language Model*. 2020. URL: <https://developer.nvidia.com/blog/openai-presents-gpt-3-a-175-billion-parameters-language-model/> (visited on 01/17/2025).
- [AW19] A. M. Antonopoulos and G. Wood: *Ethereum - Grundlagen und Programmierung*, 1st. O'Reilly, 2019. ISBN: 978-3-96009-110-3.
- [Ava25] Avalance: *SMART CONTRACTS FOR DEVELOPERS*. 2025. URL: <https://www.avax.network/smart-contracts> (visited on 03/25/2025).
- [BL05] S. Banerjee and A. Lavie: *METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments*. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. Association for Computational Linguistics, 2005, pages 65–72. URL: <https://aclanthology.org/W05-0909/>.
- [BJT+22] M. Bavarian et al.: *Efficient Training of Language Models to Fill in the Middle*. In *arXiv e-prints*, 2022. arXiv: 2207.14255v1 [cs.CL].
- [Boh23] N. Bohorquez: *Top 10 Tools For Hyperparameter Optimization In Python*. 2023. URL: <https://www.activestate.com/blog/top-10-tools-for-hyperparameter-optimization-in-python/> (visited on 01/15/2025).
- [BHA+22] R. Bommasani et al.: *On the Opportunities and Risks of Foundation Models*. In *arXiv e-prints*, 2022. arXiv: 2108.07258v3 [cs.LG].
- [BMR+20] T. B. Brown et al.: *Language models are few-shot learners*. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS '20. Curran Associates Inc., 2020. ISBN: 9781713829546. URL: <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf>.
- [But14] V. Buterin: *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. In *Ethereum.org*, 2014. URL: <https://shorturl.at/uyJ8R>.

- [CS24] R. Caballar and C. Stryker: *What are LLM benchmarks?* 2024. URL: <https://www.ibm.com/think/topics/llm-benchmarks> (visited on 10/06/2024).
- [Cha24] Chainlink: *Top 6 Smart Contract Languages in 2024*. 2024. URL: <https://chain.link/education-hub/smart-contract-programming> (visited on 03/25/2025).
- [CTJ+21] M. Chen et al.: *Evaluating Large Language Models Trained on Code*. In *arXiv e-prints*, 2021. arXiv: 2107.03374v2 [cs.LG].
- [CXZG16] T. Chen, B. Xu, C. Zhang, and C. Guestrin: *Training Deep Nets with Sublinear Memory Cost*. In *arXiv e-prints*, 2016. arXiv: 1604.06174v2 [cs.LG].
- [Dao24] T. Dao: *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. In *The Twelfth International Conference on Learning Representations*, 2024. URL: <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- [DPHZ23] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer: *QLoRA: Efficient Finetuning of Quantized LLMs*. In *Advances in Neural Information Processing Systems*. Volume 36. Curran Associates, Inc., 2023, pages 10088–10115. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/1feb87871436031bdc0f2beaa62a049b-Paper-Conference.pdf.
- [DCLT19] J. Devlin, M. Chang, K. Lee, and K. Toutanova: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pages 4171–4186. DOI: 10.18653/v1/N19-1423. URL: <https://aclanthology.org/N19-1423/>.
- [Dil24] C. Dilmegani: *Top 9 Smart Contract Use Cases & Real-life Examples in '25*. 2024. URL: <https://research.aimultiple.com/smart-contracts-examples/> (visited on 01/13/2025).
- [Doc25] GitHub Docs: *Finding existing vulnerabilities in code*. 2025. URL: <https://shorturl.at/C5hp9> (visited on 03/30/2025).
- [DFAC20] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz: *Empirical review of automated analysis tools on 47,587 Ethereum smart contracts*. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. ICSE '20*. Association for Computing Machinery, 2020, pages 530–541. ISBN: 9781450371216. DOI: 10.1145/3377811.3380364. URL: <https://doi.org/10.1145/3377811.3380364>.

- [Edi23] Metana Editorial: *Common Solidity Security Vulnerabilities & How to Avoid Them*. 2023. URL: <https://metana.io/blog/common-solidity-security-vulnerabilities-how-to-avoid-them/> (visited on 04/23/2024).
- [Edi24] Metana Editorial: *How Similar Are JavaScript And Solidity?* 2024. URL: <https://metana.io/blog/how-similar-are-javascript-and-solidity/> (visited on 10/05/2024).
- [Eth24] Ethereum: *Solidity Documentation*. 2024. URL: <https://docs.soliditylang.org/en/latest/> (visited on 01/13/2025).
- [Fac22] Hugging Face: *The Hugging Face Course, 2022*. 2022. URL: <https://huggingface.co/course> (visited on 01/18/2025).
- [FGG19] J. Feist, G. Grieco, and A. Groce: *Slither: A Static Analysis Framework for Smart Contracts*. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pages 8–15. DOI: 10.1109/WETSEB.2019.00008.
- [Fer24] J. Ferrer: *How Transformers Work: A Detailed Exploration of Transformer Architecture*. 2024. URL: <https://www.datacamp.com/tutorial/how-transformers-work> (visited on 02/07/2025).
- [FS19] T. Fertig and A. Schütz: *Blockchain für Entwickler*, 1st. Rheinwerk Verlag, 2019. ISBN: 978-3-8362-6390-0.
- [Fra18] P. I. Frazier: *A Tutorial on Bayesian Optimization*. In *arXiv e-prints*, 2018. arXiv: 1807.02811v1 [stat.ML].
- [GSM+24] D. Golovin et al.: *Google Vizier: A Service for Black-Box Optimization*. In *Google Research*, 2024. Google Vizier: A Service for Black-Box Optimization.pdf.
- [Gru21] M. Grunitz: *Rule-based AI vs machine learning: what's the difference?* 2021. URL: <https://wearebrain.com/blog/rule-based-ai-vs-machine-learning-whats-the-difference/> (visited on 03/30/2025).
- [GZY+24] D. Guo et al.: *DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence*. In *arXiv e-prints*, 2024. arXiv: 2401.14196v2 [cs.SE].
- [HSW+22] E. J. Hu et al.: *LoRA: Low-Rank Adaptation of Large Language Models*. In *International Conference on Learning Representations*, 2022. URL: <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [Iye22] A. Iyer: *Behind ChatGPT's Wisdom: 300 Bn Words, 570 GB Data*. 2022. URL: <https://analyticsindiamag.com/ai-features/behind-chatgpts-wisdom-300-bn-words-570-gb-data/> (visited on 03/25/2025).

- [JDO+17] M. Jaderberg et al.: *Population Based Training of Neural Networks*. In *arXiv e-prints*, 2017. arXiv: 1711.09846v2 [cs.LG].
- [Jav18] M. Javaheripi: *The Surprising Power of Small Language Models*. 2018. URL: <https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/> (visited on 04/24/2024).
- [Kal22] E. Kalliamvakou: *Research: quantifying GitHub Copilot’s impact on developer productivity and happiness*. 2022. URL: <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/> (visited on 01/19/2025).
- [KB15] D. P. Kingma and J. Ba: *Adam: A Method for Stochastic Optimization*. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL: <http://arxiv.org/abs/1412.6980>.
- [KBA24] D. J. Kopiczko, T. Blankevoort, and Y. M. Asano: *VeRA: Vector-based Random Matrix Adaptation*. In *The Twelfth International Conference on Learning Representations*, 2024. URL: <https://openreview.net/forum?id=NjNfLdxr3A>.
- [KR18] T. Kudo and J. Richardson: *SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2018, pages 66–71. DOI: 10.18653/v1/D18-2012. URL: <https://aclanthology.org/D18-2012/>.
- [LJR+20] L. Li et al.: *A System for Massively Parallel Hyperparameter Tuning*. In *Proceedings of Machine Learning and Systems*. Volume 2, 2020, pages 230–246. URL: <https://shorturl.at/1fN1J>.
- [LAZ+23] R. Li et al.: *StarCoder: may the source be with you!* In *Transactions on Machine Learning Research*, 2023. Reproducibility Certification. ISSN: 2835-8856. URL: <https://openreview.net/forum?id=KoFOg41haE>.
- [LL19] Petro Liashchynskyi and Pavlo Liashchynskyi: *Grid Search, Random Search, Genetic Algorithm: A Big Comparison for NAS*. In *arXiv e-prints*, 2019. arXiv: 1912.06059v1 [cs.LG].
- [Lin04] C. Lin: *ROUGE: A Package for Automatic Evaluation of Summaries*. In *Text Summarization Branches Out*, 2004. URL: <https://aclanthology.org/W04-1013/>, pages 74–81.
- [LH19] I. Loshchilov and F. Hutter: *Decoupled Weight Decay Regularization*. In *International Conference on Learning Representations*, 2019. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.

- [Mar24] B. Marie: *Fine-Tuning LLMs with 32-bit, 8-bit, and Paged AdamW Optimizers*. 2024. URL: <https://towardsdatascience.com/fine-tuning-llms-with-32-bit-8-bit-and-paged-adamw-optimizers-1034e3105634/> (visited on 02/05/2025).
- [Mas23] N. Mashette: *Small Language Models (SLMs)*. 2023. URL: <https://medium.com/@nageshmashette32/small-language-models-slms-305597c9edf2> (visited on 05/15/2024).
- [McC04] S. McConnell: *Code Complete: A Practical Handbook of Software Construction*, 2nd. Microsoft Press, 2004. ISBN: 978-0-73561-967-8.
- [Mey21] U. Meyer: *Compilerbau*, 1st. Rheinwerk Verlag, 2021. ISBN: 978-3-8362-7733-4.
- [MEBM24] G. Morello, M. Eshghie, S. Bobadilla, and M. Monperrus: *DISL: Fueling Research with A Large Dataset of Solidity Smart Contracts*. In *arXiv e-prints*, 2024. arXiv: 2403.16861v2 [cs.SE].
- [Nak08] S. Nakamoto: *Bitcoin: A Peer-to-Peer Electronic Cash System*. In *Bitcoin.org*, 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [NKQ+24] H. Naveed et al.: *A Comprehensive Overview of Large Language Models*. In *arXiv e-prints*, 2024. arXiv: 2307.06435v9 [cs.CL].
- [Nie19] M. Nielsen: *Neural Networks and Deep Learning*. 2019. URL: <http://neuralnetworksanddeeplearning.com/index.html> (visited on 01/17/2025).
- [OAA+24] OpenAI et al.: *GPT-4 Technical Report*. In *arXiv e-prints*, 2024. arXiv: 2303.08774v6 [cs.CL].
- [PRWZ02] K. Papineni, S. Roukos, T. Ward, and W. Zhu: *Bleu: a Method for Automatic Evaluation of Machine Translation*. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2002, pages 311–318. DOI: 10.3115/1073083.1073135. URL: <https://aclanthology.org/P02-1040/>.
- [PK08] N. N. Pise and P. Kulkarni: *A Survey of Semi-Supervised Learning Methods*. In *2008 International Conference on Computational Intelligence and Security*. Volume 2, 2008, pages 30–34. DOI: 10.1109/CIS.2008.204.
- [RSR+20] C. Raffel et al.: *Exploring the limits of transfer learning with a unified text-to-text transformer*. In *J. Mach. Learn. Res.*, volume 21 (1), 2020. ISSN: 1532-4435. URL: <https://www.jmlr.org/papers/volume21/20-074/20-074.pdf>.
- [RGL+20] S. Ren et al.: *CodeBLEU: a Method for Automatic Evaluation of Code Synthesis*. In *arXiv e-prints*, 2020. arXiv: 2009.10297v2 [cs.SE].

- [Res25] Restack: *Gpt-3 Training Dataset Insights*. 2025. URL: <https://www.restack.io/p/ai-dataset-creation-answer-gpt-3-training-dataset-cat-ai> (visited on 03/25/2025).
- [Ros22] M. Rossini: *Slither Audited Smart Contracts Dataset*. 2022. URL: <https://huggingface.co/datasets/mwritescode/slither-audited-smart-contracts> (visited on 12/17/2024).
- [RGG+24] B. Rozière et al.: *Code Llama: Open Foundation Models for Code*. In *arXiv e-prints*, 2024. arXiv: 2308.12950v3 [cs.CL].
- [Rud17] S. Ruder: *An overview of gradient descent optimization algorithms*. In *arXiv e-prints*, 2017. arXiv: 1609.04747v2 [cs.LG].
- [SAP24] SAP: *What is blockchain technology?* 2024. URL: <https://www.sap.com/products/artificial-intelligence/what-is-blockchain.html> (visited on 01/10/2025).
- [SN12] M. Schuster and K. Nakajima: *JAPANESE AND KOREAN VOICE SEARCH*. In *Google Inc.*, 2012. JAPANESE AND KOREAN VOICE SEARCH.pdf.
- [SHB16] R. Sennrich, B. Haddow, and A. Birch: *Neural Machine Translation of Rare Words with Subword Units*. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2016, pages 1715–1725. DOI: 10.18653/v1/P16-1162. URL: <https://aclanthology.org/P16-1162/>.
- [Sie16] D. Siegel: *Understanding The DAO Attack*. 2016. URL: <https://www.coindesk.com/learn/understanding-the-dao-attack> (visited on 04/23/2024).
- [Ski25] Skillsoft: *LLM Accuracy, Performance, and Trade-Offs*. 2025. URL: <https://shorturl.at/mHhOF> (visited on 03/24/2025).
- [SGM19] E. Strubell, A. Ganesh, and A. McCallum: *Energy and Policy Considerations for Deep Learning in NLP*. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pages 3645–3650. DOI: 10.18653/v1/P19-1355. URL: <https://aclanthology.org/P19-1355/>.
- [TLI+23] H. Touvron et al.: *LLaMA: Open and Efficient Foundation Language Models*. In *arXiv e-prints*, 2023. arXiv: 2302.13971v1 [cs.CL].
- [VSP+17] A. Vaswani et al.: *Attention is all you need*. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS’17. Curran Associates Inc., 2017, pages 6000–6010. ISBN: 9781510860964. URL: <https://shorturl.at/riQex>.

- [Ver23] A. Verma: *NLP Evaluation: Intrinsic vs. Extrinsic Assessment*. 2023. URL: <https://ai.plainenglish.io/nlp-evaluation-intrinsic-vs-extrinsic-assessment-ff1401505631> (visited on 03/30/2025).
- [WLG+23] Y. Wang et al.: *CodeT5+: Open Code Large Language Models for Code Understanding and Generation*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pages 1069–1088. DOI: 10.18653/v1/2023.emnlp-main.68. URL: <https://aclanthology.org/2023.emnlp-main.68/>.
- [XXQ+23] L. Xu et al.: *Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment*. In *arXiv e-prints*, 2023. arXiv: 2312.12148v1 [cs.CL].
- [ZLL+21] A. Zhang et al.: *Dive into Deep Learning*. 2021. URL: <https://d2l.ai/index.html> (visited on 01/17/2025).
- [ZQD+21] F. Zhuang et al.: *A Comprehensive Survey on Transfer Learning*. In *Proceedings of the IEEE*, volume 109 (1), 2021, pages 43–76. DOI: 10.1109/JPROC.2020.3004555.

Acronyms

AI Artificial Intelligence. 2, 11, 44

ASHA asynchronous successive halving. 15, 33

BERT Bidirectional Encoder Representations from Transformers. 7, 14

BLEU Bilingual Evaluation Understudy. ix, 15, 37–40, 57

BPE Byte-Pair Encoding. 13, 30

CLM Causal Language Model. 6–9, 30, 31

DAO decentralized autonomous organization. 2, 18

DApp decentralized application. 4

DeFi decentralized finance. 5, 18

EVM Ethereum Virtual Machine. 1, 4, 5

FIM fill-in-the-middle. 9, 12, 18, 27–29, 31, 39, 40, 43

GPT Generative Pretrained Transformer. 7, 8, 11

JVM Java Virtual Machine. 4

LLaMA Large Language Model Meta AI. 9, 11, 14

LLM Large Language Model. ix, 2, 3, 5, 7, 9, 11–15, 17, 18, 26, 30–32, 37, 43, 44

LoRA low rank adaptation. 14, 15, 31–33

METEOR Metric for Evaluation of Translation with Explicit ORdering. ix, 15, 37–40, 57

ML machine learning. 7, 9, 13

MLM Masked Language Model. 6–8

NFT non-fungible token. 5, 18

PEFT parameter efficient fine-tuning. 14, 30

QLoRA quantized low rank adaption. 11, 14, 31, 32

Acronyms

S2S Sequence to Sequence. 6–8

SLM Small Language Model. 14

List of Figures

2.1	Transformer architecture	6
5.1	Preprocessing pipeline	19
5.2	Validation loss of Ray Tune trials	35
5.3	Training loss of Ray Tune trials	35
5.4	Validation loss of final model	36
5.5	Training loss of final model	36

List of Tables

5.1	Number of contracts and loading time for each source	17
5.2	Result of the Slither Analysis step	23
5.3	Composition of the final dataset	29
5.4	Hyperparameter optimization results	34
6.1	BLEU and METEOR evaluation results	40

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass die vorliegende Arbeit selbstständig verfasst wurde und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt wurden.

Hamburg, am March 31, 2025

Fabian Hensel